



TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

RentoyNow

Un juego de cartas multijugador online para jugar al Rentoy
en cualquier lugar

Autor

Víctor Casalini González

Directores

Luis Gonzaga Baca Ruiz



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, junio de 2026



RentoyNow

Un juego de cartas multijugador online para jugar al Rentoy
en cualquier lugar

Autor

Víctor Casalini González

Directores

Luis Gonzaga Baca Ruiz

RentoyNow: Juego de cartas multijugador para dispositivos móviles

Víctor Casalini González

Palabras clave: Videojuego, juego de cartas, multijugador online, dispositivos móviles, Flutter, Firebase

Resumen

El objetivo principal de este proyecto es desarrollar un videojuego multijugador online para dispositivos móviles basado en el tradicional juego de cartas de baraja española “Rentoy”. La aplicación busca trasladar la experiencia del juego clásico al entorno digital, permitiendo jugar sin necesidad de cartas físicas. La mecánica central respeta las reglas originales de las partidas por equipos, donde los jugadores combinan estrategia y suerte.

Como objetivo secundario, se plantea el aprendizaje y uso del entorno de desarrollo Flutter, así como la integración de servicios en la nube con Firebase (Realtime Database, Auth y Storage) para la gestión de usuarios y la sincronización de partidas en tiempo real con latencia mínima, aplicando para ello los conocimientos adquiridos a lo largo del Grado en Ingeniería Informática.

RentoyNow: Multiplayer card game for mobile devices

Víctor Casalini González

Keywords: Video game, card game, online multiplayer, mobile devices, Flutter, Firebase

Abstract

The main objective of this project is to develop an online multiplayer video game for mobile devices based on the traditional Spanish deck card game “Rentoy”. The application aims to translate the experience of the classic game into the digital environment, allowing players to enjoy it without the need for physical cards. The core mechanics respect the original rules of team matches, where players combine strategy and luck.

As a secondary objective, the learning and use of the Flutter development environment is proposed, as well as the integration of cloud services with Firebase (Realtime Database, Auth, and Storage) for user management and real-time game synchronization with minimal latency, applying the knowledge acquired throughout the Degree in Computer Engineering.

Yo, **Víctor Casalini González**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 74540546F, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Víctor Casalini González

Granada a ___ de _____ de 2026.

D. **Luis Gonzaga Baca Ruiz**, Profesor del Área de Lenguajes y Sistemas Informáticos del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***RentoyNow: Juego de cartas multi-jugador para dispositivos móviles***, ha sido realizado bajo su supervisión por **Víctor Casalini González**, y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada, junio de 2026.

El director:

Luis Gonzaga Baca Ruiz

AGRADECIMIENTOS

A mi familia, por ser mi pilar fundamental y haberme brindado un apoyo incondicional que ha hecho posible completar con éxito esta etapa de mi vida. Me gustaría expresar un agradecimiento muy especial a mi madre; gracias por tu amor infinito y por seguir guiándome y dándome fuerzas allí donde estés, tu recuerdo sigue siendo mi mayor inspiración y motivación.

A mis amigos, tanto a los de toda la vida como a aquellos que he tenido la suerte de conocer durante la carrera. Gracias por estar ahí y por haber logrado que este camino fuera mucho más ameno y llevadero.

Por último, me gustaría agradecer a mi tutor, Luis Gonzaga Baca Ruiz, por haber aceptado dirigir este proyecto y por la ayuda prestada durante todo el proceso. Sus consejos y ánimos han sido clave para poder dar forma a este trabajo y completarlo con éxito.

Índice general

1. Introducción	1
1.1. Contexto general	1
1.2. Motivación personal	2
2. Estado del arte	3
2.1. Origen de los Naipes y Difusión Geográfica	3
2.2. La Baraja Española: Estructura, Palos y Significado Sociocul- tural	4
2.3. Características y Curiosidades de la Baraja Española	5
2.4. El Concepto de Videojuego Doméstico y la influencia Analógica	6
2.5. Los Primeros Dispositivos Portables	7
2.6. La Revolución del Smartphone	8
2.7. UNO!™	9
2.8. Balatro	10
2.9. Spades: Classic Card Game (MobilityWare)	12
2.10. La Brisca (Quarzo Apps)	12
2.11. Análisis Comparativo y Motivación del Proyecto	13
3. Objetivos, Metodología y Planteamiento	15
3.1. Objetivos	15
3.2. Metodología Ágil (Scrum)	15
3.2.1. Flujo de Trabajo Scrum	16
3.2.2. Product Backlog e Historias de Usuario	17
3.2.3. Planificación de Sprints	18
3.3. Presupuesto del Proyecto	24
3.3.1. Costes de Personal	25
3.3.2. Costes de Hardware	25
3.3.3. Costes de Software	26
3.3.4. Coste Total del Proyecto	26
3.4. Planteamiento del Problema	26
3.5. Arquitectura Multijugador y Sincronización en Tiempo Real .	27
3.6. Lógica del Juego y Motor de Reglas	27
3.7. Interfaces y Experiencia de Usuario	28
3.8. Gestión de Usuarios y Autenticación	28
3.9. Especificación de Requisitos	29
3.9.1. Requisitos Funcionales	29
3.9.2. Requisitos No Funcionales	30

4. Diseño del Sistema	33
4.1. Herramientas utilizadas	33
4.1.1. Flutter y Dart	33
4.1.2. Firebase	33
4.1.3. Git y GitHub	34
4.1.4. Jira Software	34
4.1.5. Visual Studio Code	35
4.1.6. LaTeX	35
4.2. Arquitectura de Software y Patrones de Diseño	35
4.2.1. Estructura Modular del Proyecto	35
4.2.2. Patrones de Diseño Aplicados	36
4.3. Modelo de Datos y Backend	37
4.3.1. Almacenamiento Persistente (Cloud Firestore)	38
4.3.2. Motor Multijugador (Realtime Database)	39
4.3.3. Entidades del Dominio (Modelos en Dart)	40
4.4. Diseño de Flujos y Secuencias	40
4.4.1. Flujo de Autenticación	41
4.4.2. Flujo de Matchmaking	41
4.4.3. Diseño del Motor de Rondas y Bazas	42
4.4.4. Diseño del Sistema de Envites (Apuestas)	43
5. Resolución e Implementación	45
5.1. Lógica Interna y Mecánicas del Juego	45
5.1.1. Reparto y Gestión de Turnos	45
5.1.2. Algoritmo de Evaluación de Bazas	45
5.1.3. Implementación del Sistema de Envites (Apuestas)	46
5.1.4. Inteligencia Artificial (Modo Bot)	47
6. Resultados y Pruebas	49
6.1. Resultados de la Interfaz de Usuario (UI)	49
6.1.1. Autenticación, Menú Principal y Perfil	50
6.1.2. Matchmaking: Creación, Reglas y Búsqueda de Salas	50
6.1.3. El Tablero de Juego y Elementos de Apoyo	51
6.2. Pruebas y Validación	52
6.2.1. Pruebas de Sincronización (Multijugador Real)	53
6.2.2. Pruebas de Lógica (Testing Interno)	53
6.2.3. Pruebas de Interfaz (UI/UX)	53
7. Conclusiones	55
Bibliografía	60

A. Tarjetas de Historias de Usuario	61
A.1. Gestión de Usuarios	61
A.2. Perfil	63
A.3. Creación y Unión de Partidas	64
A.4. Sala de Espera	65
A.5. Lógica de Juego	66
A.6. Envites	68
A.7. Puntuación y Rondas	69
A.8. Privacidad de Cartas	70
A.9. Historial de Bazas	71
A.10. Consulta de Reglas	71
A.11. Modo Entrenamiento	72
 B. Esquemas de Base de Datos	 75
B.1. Realtime Database (Motor del Juego)	75
B.2. Cloud Firestore (Almacenamiento Persistente)	76
B.2.1. Colección: usuarios	77
B.2.2. Colección: partidas	77

Índice de figuras

2.1. Dibujo de Thomas Allom de 1843, que muestra mujeres chinas jugando a los naipes.	3
2.2. Simbología de diferentes barajas.	4
2.3. Los “ases” de la baraja española.	5
2.4. La “pinta” de cada palo.	6
2.5. Magnavox Odyssey con sus accesorios.	7
2.6. Game boy con varios juegos.	8
2.7. Steve Jobs en la presentación del iPhone 3 en 2008.	9
2.8. Interfaz de partida de UNO!™ Mobile.	9
2.9. Captura de pantalla de una partida de Balatro.	11
2.10. Captura de pantalla las descargas de Balatro en Google Play.	11
2.11. Pantalla de juego de Spades de MobilityWare.	12
2.12. Interfaz de la aplicación La Brisca de Quarzo.	13
3.1. Diagrama de Gantt con la planificación temporal de los Sprints.	16
4.1. Diagrama de la Base de Datos (Estructura de Firestore y Real-time Database).	38
4.2. Diagrama de Secuencia del flujo de autenticación con Google y sincronización en Cloud Firestore.	41
4.3. Diagrama de Secuencia del flujo de Matchmaking (Creación y entrada por PIN).	42
4.4. Diagrama de Secuencia del lanzamiento de cartas y cálculo asíncrono de baza.	43
4.5. Diagrama de Secuencia del Sistema de Envites (Proceso de apuestas dinámicas).	43
6.1. Diagrama de Flujo de Navegación e interfaces del sistema (UX Flow).	49
6.2. De izquierda a derecha: Inicio de Sesión, Registro, Menú Principal y Perfil del Jugador.	50
6.3. Modal de creación de partida con reglas personalizadas, Sala de Espera y teclado para unirse por PIN.	51
6.4. Modal de Envite con opciones de respuesta (izq.) y Popup de Historial de cartas jugadas (der.).	52
6.5. Tapete interactivo de juego, modal de Reglas del Juego y pantalla de Fin de Partida.	52

1. INTRODUCCIÓN

1.1. Contexto general

El uso generalizado de los teléfonos inteligentes ha cambiado radicalmente la forma en la que consumimos entretenimiento. En los últimos años, los dispositivos móviles se han consolidado como la plataforma de videojuegos más accesible, superando en número de usuarios a las consolas tradicionales. Dentro de las tiendas de aplicaciones, existe una tendencia clara hacia la digitalización de juegos de mesa y de cartas clásicos.

Juegos universales como el Solitario, el Póker o el UNO! cuentan con adaptaciones que acumulan millones de descargas a nivel global. En el ámbito nacional, juegos propios de la baraja española como el Mus, el Tute o la Brisca también han dado el salto a las pantallas con gran éxito. Estas aplicaciones permiten a los usuarios disfrutar de sus pasatiempos favoritos sin necesidad de disponer de una baraja física, eliminando las barreras de la distancia y facilitando jugar con amigos o desconocidos a través de internet en cualquier momento.

Sin embargo, a pesar de esta amplia oferta, muchos juegos tradicionales con un carácter más regional o local se están quedando fuera de esta transición digital. Este es el caso del Rentoy, un juego de cartas que para muchos de nosotros es mucho más que un simple entretenimiento: es parte de nuestra identidad y cultura en lugares como la costa de Granada y la Alpujarra, siendo la excusa perfecta para pasar horas compartiendo con amigos y familia.

Los juegos de cartas de la baraja española, como el Rentoy, representan una parte fundamental del patrimonio cultural y social en diversas regiones. El desarrollo de una versión digital no solo responde a una necesidad de ocio contemporáneo, sino que también actúa como una herramienta para preservar estas tradiciones y acercarlas a las nuevas generaciones. Desde el punto de vista técnico, la creación de este tipo de aplicaciones fomenta el desarrollo de competencias en ingeniería de software, como la gestión de estados complejos y la sincronización de datos en tiempo real.

1.2. Motivación personal

Siempre me han apasionado los videojuegos y esa fue la razón principal por la que decidí estudiar Ingeniería Informática. Sin embargo, para mi Trabajo de Fin de Grado, no quería limitarme a desarrollar una aplicación cualquiera; buscaba algo que tuviera un sentido real para mí y que pudiera aportar algo a mi entorno más cercano.

La elección del Rentoy como tema para este Trabajo de Fin de Grado nace de mi vinculación personal con este juego de cartas, muy jugado en la zona donde vivo y el cual ha sido protagonista de innumerables momentos con familiares y amigos. En muchas ocasiones, al querer jugar, nos encontrábamos con el inconveniente de no disponer de los elementos físicos necesarios o de la imposibilidad de reunirnos presencialmente.

Al observar que no existían opciones digitales para solventar estas situaciones y permitir el juego a distancia, decidí emprender el reto de crear “RentoyNow”, uniendo así una tradición local con los conocimientos técnicos desarrollados a lo largo de la carrera.

2. ESTADO DEL ARTE

Los juegos de cartas constituyen uno de los ejemplos más tempranos de lo que podría denominarse “juego lúdico portátil”. Su historia es una crónica de difusión cultural, adaptación social y simplicidad estructural, capaz de trascender siglos y geografías.

2.1. Origen de los Naipes y Difusión Geográfica

La gran mayoría de las alusiones históricas al origen de los naipes, citan China como el lugar de procedencia, concretamente en las llamadas cartas de Ma-Diao (siglo IX). Realizándose después, una adaptación de estas cartas con otras, a partir del dominó chino y el significado simbólico del papel-moneda.

Existen hipótesis que señalan la India como otro posible origen de los naipes. En su juego de cartas de los Diez Avatares, se plasman en 10 palos, las encarnaciones de la diosa hindú Vishnú.



Figura 2.1: Dibujo de Thomas Allom de 1843, que muestra mujeres chinas jugando a los naipes.

Más tarde, una mezcla de ambas influencias, llegaron a través de la Ruta de la Seda al mundo islámico y posteriormente a Europa. Es sorprendente comprobar, como los naipes se amoldaron a diferentes entornos y circuns-

tancias. Adaptando su simbología y figuras a las características del lugar, dejando de manifiesto la universalidad de su método: reglas sencillas y claras, pocos elementos y turnos para ponerlas en práctica.

ALEMANIA	Corazones 	Cascabeles 	Bellotas 	Hojas 
FRANCIA	Corazones 	Diamantes 	Tréboles 	Picas 
ESPAÑA E ITALIA	Copas 	Oros 	Bastos 	Espadas 

Figura 2.2: Simbología de diferentes barajas.

Etimológicamente, la palabra “naipe” proviene del árabe “nā’ib” (figura, representante), usada para hablar de la forma islámica “mameluca” de los cuatro palos de la baraja y las figuras de la corte (teniente, virrey y emir) [1, 2].

2.2. La Baraja Española: Estructura, Palos y Significado Sociocultural

La aparición del naipe en Europa y su adecuación a las diferentes realidades geográficas e históricas, dio origen a formatos locales muy variados. En nuestro país (finales del S. XIV y principios del S.XV) se afianzó lo que hoy llamamos “baraja española” constituida por cuatro palos: oros, copas, espadas y bastos [3]. La baraja de más antigüedad que se conserva en España, data de 1390, ubicada en Sevilla y ya utilizaba esta simbología [1].

Pero esta simbología no fue accidental y meramente decorativa, cada palo de la baraja española, representaba los diferentes estatus de la sociedad medieval existente en la época. Así, los “Oros” representan al comercio y el dinero; las “Copas” a la Iglesia; las “Espadas” simbolizaban a la nobleza y a los militares y los “Bastos” a la mayor parte de la sociedad feudal de la época, el campesinado [3].

Este simbolismo hacía de la baraja una representación lúdica de la estructura feudal, en la que el entretenimiento servía como espejo de las jerarquías sociales y valores de su tiempo. De este modo, la baraja española no sólo



Figura 2.3: Los “ases” de la baraja española.

consolidó un modelo de juego, sino que se transformó en un instrumento cultural que reflejaba los imaginarios sociales y económicos de la época [4].

2.3. Características y Curiosidades de la Baraja Española

La baraja más frecuente es aquella que consta de 48 cartas; 3 figuras por cada “familia” o “palo” y 9 numeradas. Sin embargo, tampoco es raro encontrar barajas españolas de 40 cartas, en las que no hay nueves ni ochos.

En comparación con otras barajas del continente europeo, la española tiene algunas peculiaridades: las figuras se representan de cuerpo entero; la figura del caballo sustituye a la de la dama o reina y con frecuencia sus cartas numéricas solo llegan a 7 [5].

El diseño que conocemos actualmente fue realizado por Heraclio Fournier (1870) en Vitoria y fue premiado en la Exposición Universal de París. Existen muchas curiosidades de esta baraja, pero destacaremos dos características que, a veces, nos pasan desapercibidas:

1. La “mirada” de las figuras del mismo palo, siempre miran hacia el mismo lado. Oros y Copas hacia la izquierda, mientras que Bastos y

Espadas lo hacen a la derecha.

2. Cada carta tiene en los bordes unas líneas, llamadas “pintas” y son diferentes para cada palo, tanto en la parte inferior, como en la superior. Los Oros tienen la línea completa; las Copas un espacio que divide la parte superior e inferior que divide la línea completa que rodea la ilustración. Las Espadas tiene dos espacios, mientras que los Bastos presentan tres espacios [6].

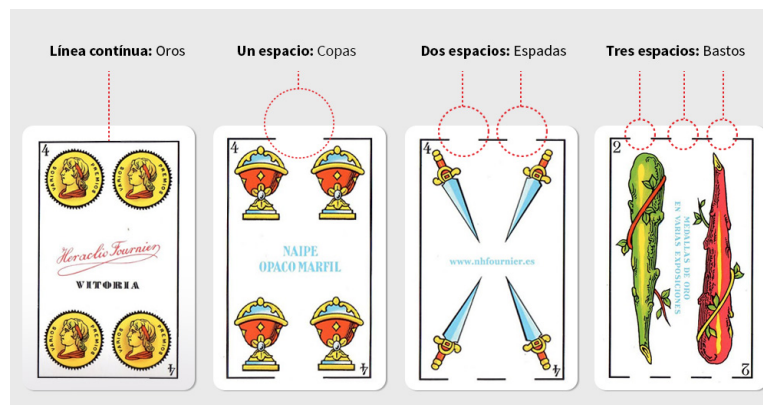


Figura 2.4: La “pinta” de cada palo.

Los juegos de cartas se pueden agrupar en tipos de juegos de cartas específicos: Juegos de bazas; de descarte; de emparejar; de solitario; de comparación; bancarios; de apuestas; de pesca; de capturar y juegos de cartas coleccionables [7].

2.4. El Concepto de Videojuego Doméstico y la influencia Analógica

Los primeros videojuegos domésticos no nacieron por “generación espontánea”, sino que fueron una revisión tecnológica de juegos ya existentes, con tradición e interacción social como los juegos de cartas o mesa. Por esto, la industria del videojuego surge en el siglo XX como una extensión tecnológica de las dinámicas lúdicas tradicionales. Uno de los acontecimientos que marcaron el comienzo de esta industria, fue la Magnavox Odyssey (1972), reconocida como la primera consola doméstica comercial [8]. mesa, marcó el comienzo del entretenimiento digital [9]. La Odyssey fue una consola pionera que contó con un total de 28 juegos en toda su vida útil [8].



Figura 2.5: Magnavox Odyssey con sus accesorios.

Resulta especialmente relevante que la Odyssey incluyera accesorios analógicos —dados, fichas, dinero de papel e incluso cartas— para complementar la innovación tecnológica [9]. Al conectar el videojuego con las dinámicas tradicionales, la consola actuó como un puente entre lo analógico y lo digital, legitimando los videojuegos como una evolución natural del ocio [10].

2.5. Los Primeros Dispositivos Portables

Al mismo tiempo que el desarrollo de las consolas domésticas, surgió la necesidad de poderlos llevar de un sitio a otro (portabilidad). En la década de los años setenta, empresas como Mattel lanzaron dispositivos electrónicos como Auto Race (1976), uno de los primeros videojuegos de mano con pantalla LED. Poco después, en 1979, la Milton Bradley Microvision introdujo el concepto de cartuchos intercambiables, sentando las bases de las consolas portables [10].

Pero, la verdadera revolución llegó con la Nintendo Game Boy (1989), un dispositivo que aunaba precio, duración de batería, resistencia y catálogo de juegos, convirtiéndose en un fenómeno global con más de 100 millones de unidades vendidas [11, 12]. La Game Boy consolidó el concepto moderno de juego portátil.



Figura 2.6: Game boy con varios juegos.

2.6. La Revolución del Smartphone

El surgimiento de los smartphones a finales de los años 2000, modificó radicalmente el concepto de portabilidad. La combinación de pantalla táctil, conectividad constante y acceso a tiendas de aplicaciones dio lugar a un nuevo ecosistema donde los videojuegos se convirtieron en experiencias accesibles desde cualquier lugar [13]. Esta transición marcó el inicio de lo que académicamente se conoce como la “revolución casual”, ampliando el público jugador mucho más allá de su nicho tradicional [14].

Anteriormente, el software móvil era limitado y preinstalado. Esto cambió con la App Store de Apple, lanzada junto al iPhone 3G en julio de 2008, que introdujo un modelo de distribución digital global [15]. Desde ese momento, los desarrolladores podían publicar directamente sus juegos sin intermediarios, dando oportunidades de acceso al mercado y abriendo paso a modelos como el “free-to-play” y las compras dentro de la aplicación [15].

En la década de 2010, la expansión del hardware móvil –permitiendo que las propiedades de los juegos móviles aumentará– el paso de redes 3G a 4G y posteriormente a 5G y la cada vez más abundante adopción mundial de smartphones, consolidaron definitivamente los videojuegos móviles como una de las principales plataformas de ocio digital hasta el punto de competir con plataformas de consolas tradicionales, transformando profundamente el modelo de negocio y la economía de la industria del entretenimiento [16].

Otro factor decisivo fue el avance de las comunicaciones, que hizo posible la proliferación de títulos multijugador online [17]. Aunque esta funcionalidad se integró inicialmente de forma contenida, quedó claro que la conectividad



Figura 2.7: Steve Jobs en la presentación del iPhone 3 en 2008.

era la dirección futura del mercado.

Este fenómeno refleja una línea evolutiva clara: desde los naipes de papel hasta los juegos digitales en la palma de la mano, el ser humano ha buscado siempre un entretenimiento accesible, portátil y socialmente significativo.

2.7. UNO!™

UNO!™ es un juego para dispositivos móviles que adapta el mítico juego de cartas creado en 1971 por Merle Robbins, un barbero de Ohio [18]. Fue Mattel163, quien lo lanzó globalmente para dispositivos Android e iOS en 2019.



Figura 2.8: Interfaz de partida de UNO!™ Mobile.

En la actualidad cuenta con más de 100 millones de descargas y es de los juegos de cartas más populares en las tiendas de aplicaciones. El juego mantiene las reglas tradicionales, pero adapta estas a un entorno online multijugador masivo.

Además, introduce mecánicas de retención modernas como avatares, un sistema de comunicación rápido mediante emojis interactivos, salas personalizadas para jugar con amigos y torneos competitivos. Al ser “free to play”, abre su accesibilidad a millones de personas, permitiendo jugar desde casa en cualquier momento sin la necesidad de reunirse físicamente.

Este juego se menciona en el trabajo para poner de manifiesto la gran dimensión y popularidad que puede llegar a adquirir un juego de cartas tradicional en un entorno digital. También, porque ha sido la principal inspiración para desarrollar nuestro juego, tanto en el aspecto visual como en la gestión estructurada de partidas.

2.8. Balatro

El 19 de febrero de 2024 salió a la luz Balatro, un juego de cartas multijugador del género “roguelike” (donde si pierdes una partida tienes que volver a comenzar desde el principio). Fue desarrollado por LocalTrunk, un creador independiente que no se esperaba que su juego triunfará. La crítica le aclamó, consiguiendo un 90 en “Metacritic”, una puntuación altísima. Siguiendo la misma línea para las demás críticas y un 97 % de reseñas positivas de los jugadores. Tal es así, que consiguió vender 119 000 copias el día de lanzamiento y llegó a estar nominado para los GOTY (Game Of The Year) [19, 20].

El juego se inspira en las reglas del póker, pero no juegas contra otros jugadores sino contra ti mismo. No requiere saber jugar al póker; lo único que necesitas es saber qué mano da más puntuación, cosa que se puede consultar en la partida.

Cada fase cuenta con tres rondas: una ciega pequeña, una ciega grande y un jefe. Cada ronda requiere llegar a una puntuación cada vez más alta y cada jefe dispone de una habilidad que impone restricciones en tus jugadas, por lo que es más difícil llegar a la puntuación requerida. Tenemos un número máximo de manos que podemos jugar (5 manos base) y también contamos con descartes (3 descartes), donde podemos cambiar hasta 5 cartas de nuestra mano.



Figura 2.9: Captura de pantalla de una partida de Balatro.

Al final de cada ronda se nos pagará una cuota base más unos extras si lo hemos hecho bien, y se abrirá el apartado de tienda donde podremos comprar Jokers, que nos darán un efecto pasivo o activo que altera la puntuación o el funcionamiento de nuestras jugadas (podemos llevar hasta 5 Jokers); cartas de planeta, que mejoran la puntuación de una mano; cartas de tarot, que sirven para mejorar cartas del mazo haciéndolas más potentes o con efectos especiales; cartas espectrales, que otorgan beneficios a cambio de algún tipo de riesgo; y, por último, los vouchers, que funcionan como mejoras permanentes (descuentos, aumento del espacio de Jokers, más recompensas, etc.).

Todo esto dota a este juego de una gran variedad de estrategias (cambiando cómo y cuánto puntuamos), y la aleatoriedad de la tienda hace que cada partida sea completamente distinta a la anterior, por lo que tendrás que adaptarte a lo que puedas comprar [21].



Figura 2.10: Captura de pantalla las descargas de Balatro en Google Play.

Balatro se menciona en el proyecto debido a su gran popularidad y relevancia en el contexto actual, lo que nos reafirma en la idea del gran interés que sigue despertando en la actualidad el género de los juegos de cartas.

2.9. Spades: Classic Card Game (MobilityWare)

Desarrollado por MobilityWare, esta aplicación es una de las adaptaciones más exitosas del clásico juego de bazas “Picas” (Spades) [22]. Con más de diez millones de descargas en Google Play, destaca por haber sabido trasladar un juego tradicional de estrategia y engaño a una interfaz limpia, fluida y perfectamente adaptada a dispositivos móviles.



Figura 2.11: Pantalla de juego de Spades de MobilityWare.

El juego cuenta con modos fuera de línea contra la inteligencia artificial, tutoriales interactivos para aprender a jugar desde cero y funciones como pistas y la opción de deshacer movimientos. Este enfoque centrado en la usabilidad nos demuestra cómo se debe guiar al jugador cuando se trata de juegos de bazas con reglas que pueden resultar complejas para los principiantes, algo vital para el diseño de interfaces en juegos de este tipo.

2.10. La Brisca (Quarzo Apps)

La Brisca (conocida en Italia como *Briscola*) es un juego de bazas de gran tradición mediterránea. Su origen histórico, aunque debatido, se rastrea frecuentemente hasta el antiguo juego francés *Brusquemille*, documentado por primera vez a principios del siglo XVIII, el cual evolucionó posteriormente al *Brisque* [23]. Rápidamente se popularizó en España, jugándose con la clásica

baraja española de 40 cartas (excluyendo ochos y nueves), donde el As y el Tres ostentan el mayor valor. Su principal atractivo radica en que no es obligatorio asistir al palo de salida, lo que le dota de una gran flexibilidad táctica y lo ha convertido en un pilar cultural del ocio en nuestro país.

Centrando el foco en la digitalización de esta tradición, destaca de forma sobresaliente “La Brisca” de Quarzo Aplicaciones y Juegos [24]. Esta aplicación ha logrado una inmensa popularidad al ofrecer una experiencia muy fiel al juego tradicional de bazas español. Permite a los usuarios jugar con la clásica baraja española de Fournier en alta definición, configurar el tamaño de las cartas y establecer reglas personalizadas.



Figura 2.12: Interfaz de la aplicación La Brisca de Quarzo.

Cuenta con un modo multijugador en línea fluido y un modo de un jugador contra bots con diferentes niveles de dificultad. Su simplicidad visual y la optimización de los recursos gráficos sirven de gran ejemplo sobre cómo mantener la esencia purista de los juegos de baraja española sin sobrecargar la pantalla del teléfono móvil.

2.11. Análisis Comparativo y Motivación del Proyecto

Todo el recorrido expuesto en este capítulo, desde la evolución histórica de las cartas y los videojuegos hasta el análisis de aplicaciones contemporáneas (UNO!™, Balatro, Spades y Brisca), convergen de manera directa en la motivación principal de este proyecto: el desarrollo de **RentoyNow**.

Mientras que juegos como UNO!TM o Spades cuentan con adaptaciones digitales de gran alcance internacional, y juegos de baraja española como la Brisca o el Mus tienen múltiples alternativas muy pulidas en las tiendas de aplicaciones, existen otros juegos tradicionales profundamente arraigados en la cultura popular de ciertas regiones que aún no han dado el salto al formato digital multijugador. Este es el caso del **Rentoy**.

El análisis de estas aplicaciones nos proporciona las claves fundamentales de lo que debe incluir el proyecto para tener éxito y resultar atractivo:

- La **fluidez y conectividad** que conseguimos observar en *UNO!*TM.
- La **claridad en las reglas** y usabilidad de *Spades*.
- La **fidelidad visual** a la baraja española como logra *La Brisca*.
- La **composición de la app** y el óptimo **aprovechamiento del espacio visual**, características esenciales para adaptar la información a los reducidos espacios de los dispositivos móviles.

En definitiva, *RentoyNow* nace para llenar este hueco en el mercado, digitalizando un juego de bazas y engaño con un fuerte componente social, permitiendo preservar la tradición del Rentoy mediante tecnología moderna (Flutter y Firebase) y acercando el juego a las nuevas generaciones a través de la plataforma en la que consumen su ocio mayoritariamente: el teléfono móvil.

3. OBJETIVOS, METODOLOGÍA Y PLANTEAMIENTO

3.1. Objetivos

El objetivo principal de este proyecto es el desarrollo integral de un videojuego multijugador online para dispositivos móviles basado en el juego de cartas tradicional “Rentoy”. Para ello, se seleccionará un framework de desarrollo móvil multiplataforma, junto con servicios de infraestructura en la nube para la gestión del backend y la sincronización en tiempo real.

Como objetivos secundarios y técnicos se plantean los siguientes puntos:

- Implementar una arquitectura de software modular y escalable, siguiendo principios de Clean Architecture para separar la lógica de negocio de la interfaz de usuario.
- Integrar un sistema de sincronización en tiempo real mediante Firebase Realtime Database para gestionar las partidas multijugador sin latencia perceptible.
- Desarrollar un sistema de autenticación de usuarios y perfiles persistentes.
- Aplicar conocimientos adquiridos durante el grado en áreas como estructuras de datos (para la gestión de la baraja), algoritmos de lógica de juego y diseño de interfaces de usuario adaptativas.
- El aprendizaje autodidacta de la tecnología Flutter y el lenguaje Dart, dado que representan un entorno de desarrollo usado con anterioridad pero no de forma exhaustiva.

3.2. Metodología Ágil (Scrum)

La metodología de este Trabajo de Fin de Grado seguirá el marco de trabajo ágil Scrum [25]. Se ha elegido esta opción por su capacidad de adaptación durante el desarrollo, ya que los proyectos de software suelen tener

bastante incertidumbre al principio. En el caso de un videojuego multijugador, donde la experiencia del usuario y la respuesta del servidor mandan, Scrum permitirá ir ajustando la interfaz y las mecánicas a medida que se avance en el código.

Para tener una visión global de la temporización antes de entrar en detalle, se presenta a continuación el cronograma general estimado para el proyecto:

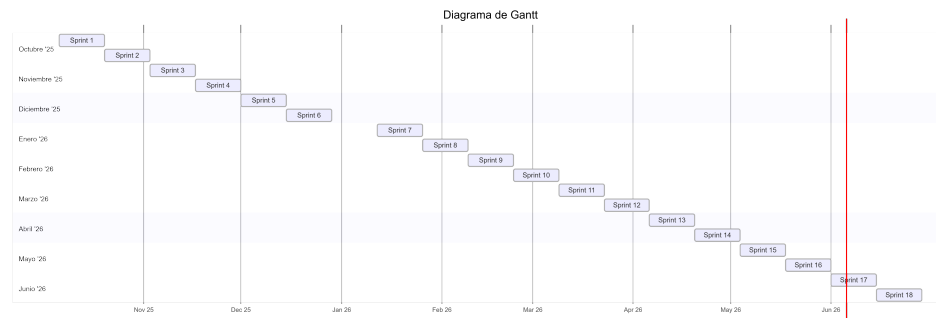


Figura 3.1: Diagrama de Gantt con la planificación temporal de los Sprints.

3.2.1. Flujo de Trabajo Scrum

El desarrollo se estructurará en las cuatro fases habituales de Scrum:

1. **Visión del Producto:** Primero se definirán los objetivos (digitalizar el Rentoy), el público (jugadores casuales) y las funciones básicas.
2. **Creación del Product Backlog:** Se documentarán todas las necesidades del juego como historias de usuario, desde el registro hasta la lógica de las cartas.
3. **Priorización del Backlog:** Los elementos se ordenarán según sus dependencias técnicas. Por ejemplo, será necesario tener un sistema de turnos antes de programar cómo jugar una carta.
4. **Sprint Planning:** Al inicio de cada Sprint se elegirán las tareas a realizar, marcando un *Sprint Goal* y creando el *Sprint Backlog*.

3.2.2. Product Backlog e Historias de Usuario

El *Product Backlog* agrupará todas las necesidades del juego traducidas en Historias de Usuario (HU). Cada historia describirá una funcionalidad desde la perspectiva del usuario y se asociará a un Requisito Funcional (RF) concreto del sistema. Dado el gran volumen de historias de usuario (40 en total), a continuación se presenta una tabla resumen. El detalle completo de cada una de ellas, incluyendo su descripción y criterios de aceptación en formato de tarjeta, se documentará íntegramente en el **Apéndice A**:

Cuadro 3.1: Product Backlog: Historias de Usuario completas.

ID	Historia de Usuario	Prioridad	SP	Sprint	RF
HU-01	Registro con email	Alta	5	Sprint 4	RF-01
HU-02	Inicio de sesión	Alta	3	Sprint 4	RF-01
HU-03	Inicio de sesión con Google	Media	5	Sprint 4	RF-01
HU-04	Recuperación de contraseña	Media	3	Sprint 4	RF-01
HU-05	Cambiar nombre de usuario	Baja	2	Sprint 5	RF-02
HU-06	Cambiar avatar	Baja	3	Sprint 5	RF-02
HU-07	Cambiar contraseña	Media	2	Sprint 5	RF-02
HU-08	Crear sala	Muy Alta	5	Sprint 6	RF-03
HU-09	Configurar máximo de jugadores	Media	2	Sprint 6	RF-03
HU-10	Generar PIN	Alta	3	Sprint 6	RF-03
HU-11	Unirse mediante PIN	Muy Alta	5	Sprint 6	RF-04
HU-12	Validar sala completa	Alta	3	Sprint 6	RF-04
HU-13	Mostrar jugadores conectados	Alta	5	Sprint 6	RF-05
HU-14	Elegir equipo	Alta	3	Sprint 6	RF-05
HU-15	Iniciar partida	Muy Alta	3	Sprint 6	RF-05
HU-16	Mostrar estado de preparación	Media	2	Sprint 6	RF-05
HU-17	Repartir cartas	Muy Alta	5	Sprint 8	RF-06
HU-18	Jugar carta	Muy Alta	8	Sprint 8	RF-06
HU-19	Validar jugada legal	Muy Alta	13	Sprint 12	RF-06
HU-20	Determinar ganador de baza	Muy Alta	8	Sprint 9	RF-06
HU-21	Controlar turnos	Muy Alta	8	Sprint 10	RF-06
HU-22	Realizar envite	Alta	8	Sprint 11	RF-07
HU-23	Aceptar envite	Alta	3	Sprint 11	RF-07
HU-24	Rechazar envite	Alta	3	Sprint 11	RF-07
HU-25	Mostrar notificaciones envite	Media	5	Sprint 11	RF-07
HU-26	Calcular puntuación	Alta	5	Sprint 9	RF-08
HU-27	Reiniciar mesa	Alta	3	Sprint 9	RF-08
HU-28	Detectar ganador de partida	Alta	3	Sprint 9	RF-08

ID	Historia de Usuario	Prioridad	SP	Sprint	RF
HU-29	Mostrar marcador	Media	2	Sprint 9	RF-08
HU-30	Ocultar mano	Media	2	Sprint 14	RF-09
HU-31	Mostrar mano nuevamente	Baja	1	Sprint 14	RF-09
HU-32	Consultar historial de bazas	Media	5	Sprint 15	RF-10
HU-33	Registrar cartas jugadas	Alta	3	Sprint 15	RF-10
HU-34	Ver reglas oficiales	Baja	3	Sprint 14	RF-11
HU-35	Consultar jerarquía de cartas	Baja	2	Sprint 14	RF-11
HU-36	Consultar envites	Baja	2	Sprint 14	RF-11
HU-37	Iniciar modo entrenamiento	Media	5	Sprint 15	RF-12
HU-38	Jugar contra IA (Bot)	Muy Alta	21	Sprint 15	RF-12
HU-39	Aprender reglas en práctica	Baja	5	Sprint 15	RF-12
HU-40	Reiniciar entrenamiento	Baja	2	Sprint 15	RF-12

3.2.3. Planificación de Sprints

El proyecto se dividirá en iteraciones de dos semanas llamadas *Sprints*. Cada uno tendrá metas específicas y entregables funcionales que permitirán evaluar el progreso y aplicar mejoras a lo largo del tiempo. El calendario tendrá en cuenta los periodos de exámenes y vacaciones de la universidad.

Los Sprints planificados son los siguientes:

Sprint 1: Investigación y viabilidad

6 de octubre - 20 de octubre de 2025

Durante este primer Sprint se llevará a cabo la investigación inicial y la configuración del entorno de trabajo.

- **Configuración de herramientas:** Creación del repositorio en GitHub y configuración del entorno con Flutter, Firebase y VSCode.
- **Investigación tecnológica:** Evaluación de la viabilidad de Firebase Realtime Database para la sincronización de un juego en tiempo real.
- **Memoria. Borrador:** Se propondrá la estructura básica de la memoria del TFG con los capítulos principales y se iniciará la redacción.

Sprint 2: Estado del Arte y Referencias

20 de octubre - 3 de noviembre de 2025

En esta fase el objetivo principal será fundamentar las bases teóricas del proyecto y las referencias del mercado.

- **Memoria. Estado del Arte:** Búsqueda de referentes teóricos sobre la evolución de los naipes, la Baraja Española y juegos tradicionales similares como la Brisca o el Tute.
- **Análisis de referentes digitales:** Estudio de juegos de cartas digitales (Balatro, Spades) para analizar sus interfaces y mecánicas.

Sprint 3: Definición de Requisitos y Arquitectura

3 de noviembre - 17 de noviembre de 2025

Durante este Sprint nos centraremos en planificar las necesidades básicas y la arquitectura del código.

- **Memoria. Requisitos:** Definición formal de los requisitos funcionales y no funcionales del sistema y los casos de uso principales.
- **Memoria. Patrones de diseño:** Investigación de los patrones a utilizar, optando por una estructura Feature-First y servicios Singleton.
- **Arquitectura en la nube:** Diseño preliminar del árbol JSON para Realtime Database y las colecciones en Firestore.

Sprint 4: Configuración inicial y Autenticación

17 de noviembre - 1 de diciembre de 2025

Este Sprint marcará el inicio del desarrollo técnico puro, centrándose en la identidad del usuario.

- **Integración Firebase:** Conexión de la app Flutter con los servicios de Google Cloud y Firebase Authentication.
- **Inicio de sesión:** Implementación de las pantallas y el flujo completo de registro e inicio de sesión seguro.
- **Memoria. Análisis:** Se comenzarán a trazar los diagramas de actividad y las historias de usuario de las funcionalidades principales.

Sprint 5: Gestión de Perfiles y Avatares

1 de diciembre - 15 de diciembre de 2025

Durante este Sprint se abordará la personalización del jugador y el almacenamiento en la nube.

- **Pantalla de perfil:** Diseño y desarrollo de la interfaz donde el jugador podrá ver y editar sus datos.
- **Sistema de avatares:** Implementación del selector de avatares, vinculando la selección de forma persistente a Firestore.
- **Búsqueda de recursos (assets):** Selección e integración de los iconos e imágenes que representarán a los distintos jugadores.

Sprint 6: Sistema de Salas (Matchmaking) y Lobby

15 de diciembre - 29 de diciembre de 2025

El objetivo de este Sprint será crear el núcleo de conectividad multijugador para enlazar a los usuarios.

- **Creación y unión de partidas:** Lógica para generar códigos PIN y permitir a otros dispositivos conectarse a la misma sala.
- **Sala de espera (Lobby):** Interfaz interactiva donde los jugadores seleccionarán equipo y el anfitrión configurará la partida.
- **Memoria. Revisión:** Revisión de los apartados redactados hasta la fecha antes del parón vacacional.

Sprint 7: Modelado de Cartas y Baraja

12 de enero - 26 de enero de 2026

Tras el descanso vacacional, el trabajo se reanudará centrándose en el dominio y las reglas básicas.

- **Modelos de datos:** Programación de las clases internas para la Baraja Española y las Cartas en lenguaje Dart.
- **Interfaz del tablero:** Creación de los primeros componentes visuales del tablero de juego (HUD) y la mesa.
- **Memoria. Resolución del trabajo:** Se empezará a documentar la metodología seguida y las herramientas a utilizar.

Sprint 8: Tablero y Primeras Mecánicas de Juego

26 de enero - 9 de febrero de 2026

Durante este Sprint se dará vida al tablero, permitiendo la primera interacción real de juego.

- **Visualización de manos:** Lógica para repartir las cartas al inicio de la ronda y mostrarlas en la parte inferior de la pantalla.
- **Lanzamiento de cartas:** Desarrollo de la mecánica para arrastrar y soltar (o pulsar) una carta, enviando la acción a Firebase.
- **Gestión del centro:** Sincronización para que todos los jugadores vean la carta jugada en el centro de la mesa al instante.

Sprint 9: Motor de Reglas y Puntuación

9 de febrero - 23 de febrero de 2026

El objetivo principal será automatizar el control de la partida sin árbitros humanos.

- **Evaluación de bazas:** Algoritmo que determinará qué carta es la ganadora teniendo en cuenta la muestra y el palo de salida.
- **Gestión de rondas:** Lógica para limpiar la mesa, asignar el minipunto al equipo ganador y repartir nuevas cartas.
- **Puntuación automática:** Cálculo en tiempo real de la puntuación final tras alcanzar el límite establecido.

Sprint 10: Sistema estricto de Turnos

23 de febrero - 9 de marzo de 2026

En este Sprint nos centraremos en robustecer la sincronización y la justicia del juego.

- **Manejador de turnos:** Implementación de bloqueos visuales y lógicos para impedir que un usuario juegue si no es su turno activo.
- **Indicadores visuales:** Añadido de feedback visual en la interfaz para marcar claramente a quién le toca jugar.

- **Resolución de conflictos:** Prevención de condiciones de carrera en Firebase mediante transacciones atómicas.

Sprint 11: Implementación de Envites y Cánticos

9 de marzo - 23 de marzo de 2026

El objetivo principal de este Sprint será integrar las mecánicas comunicativas propias del Rentoy.

- **Cánticos:** Lógica para proponer un envite a los rivales (subir la apuesta de puntos de la ronda actual).
- **Interrupción del flujo:** Mecanismo asíncrono que pausará la ronda para todos los clientes hasta que el equipo rival acepte o rechace.
- **Nuevos botones UI:** Interfaces emergentes para responder a los envites rápidamente sin entorpecer el tablero.

Sprint 12: Mecánica de Arrastre

23 de marzo - 6 de abril de 2026

En este Sprint se abordará el desafío algorítmico más complejo: la regla del arrastre.

- **Validación de tiradas:** Algoritmo que forzará a jugar el palo de salida si se posee, y priorizará la jerarquía de la muestra.
- **Feedback de error:** Implementación de avisos visuales (Snackbars) para informar al jugador si intenta jugar una carta ilegal.
- **Memoria. Diseño:** Diagramación de las clases y la arquitectura interna para adjuntarla al documento final.

Sprint 13: Partidas Personalizadas e Invitaciones

6 de abril - 20 de abril de 2026

El objetivo de este Sprint será expandir la jugabilidad y el aspecto social.

- **Reglas dinámicas:** Opciones al crear la sala para modificar el límite de puntos o deshabilitar ciertas cartas.

- **Sistema de invitaciones:** Creación de botones rápidos para invitar a amigos a la sala activa.
- **Memoria. Resultados y Pruebas:** Inicio de la redacción de este apartado comprobando los flujos implementados.

Sprint 14: Pulido de Interfaz y Tutoriales

20 de abril - 4 de mayo de 2026

Este Sprint estará dedicado a la experiencia de usuario (UX/UI).

- **Animaciones:** Integración de animaciones suaves al lanzar y recoger cartas de la mesa.
- **Overlay de Reglas:** Creación de un menú flotante accesible en plena partida para enseñar a jugar a usuarios principiantes.
- **Responsividad:** Ajustes para garantizar que el tablero se vea correctamente en distintos tamaños de pantallas móviles.

Sprint 15: Modo Bot y Registro de Historial

4 de mayo - 18 de mayo de 2026

En esta fase se incorporarán funcionalidades que no requieran de otros jugadores.

- **Modo Práctica (Bot):** Implementación de un sistema básico de IA simple para simular rivales y jugar offline.
- **Historial de partidas:** Lógica para guardar un listado de los identificadores de las salas jugadas dentro del perfil del jugador.

Sprint 16: Seguridad y Limpieza en la Nube

18 de mayo - 1 de junio de 2026

El objetivo de este Sprint será blindar la aplicación de cara a un entorno de producción.

- **Security Rules:** Restricción de lectura y escritura en la base de datos de Firebase para proteger la privacidad.

- **Garbage Collection:** Script descentralizado en los clientes para borrar automáticamente las salas con más de 2 horas de antigüedad.
- **Memoria. Conclusiones:** Redacción de los resultados obtenidos y reflexiones finales sobre los objetivos cumplidos.

Sprint 17: Fase de Pruebas y Arreglo de Errores

1 de junio - 15 de junio de 2026

Sprint íntegramente dedicado a la calidad y estabilización.

- **QA y Testing:** Pruebas exhaustivas con múltiples dispositivos reales conectados simultáneamente.
- **Resolución de bugs:** Corrección de errores de sincronización menores, optimización del uso de memoria y mejora de latencia.
- **Memoria. Pruebas:** Documentación gráfica (capturas) de las pruebas a realizar y métricas de uso.

Sprint 18: Documentación Final y Preparación de Defensa

15 de junio - 29 de junio de 2026

El Sprint final se dedicará exclusivamente al cierre académico.

- **Memoria. Revisión Final:** Corrección ortotipográfica y compilación final del documento LaTeX completo.
- **Presentación:** Elaboración de las diapositivas resumiendo el trabajo para el tribunal.
- **Exposición Oral:** Ensayo de la defensa y preparación del discurso y la demostración técnica en vivo.

3.3. Presupuesto del Proyecto

Cualquier proyecto de software tiene costes. Aunque este Trabajo de Fin de Grado es puramente académico, se ha calculado un presupuesto estimado a priori como si se tratara de un encargo profesional. Los costes contemplan el personal, el hardware y el software a utilizar.

3.3.1. Costes de Personal

Para el cálculo de los costes de personal se han definido dos perfiles fundamentales para el desarrollo del proyecto: un Desarrollador (encargado de la implementación) y un Director de Proyecto/Consultor (el tutor, encargado de la supervisión y validación).

El salario del desarrollador se ha fijado en 20 euros por hora, de acuerdo con la media estipulada para perfiles *Junior/Full-Stack* en portales de empleo del sector IT español para el periodo de desarrollo. Por su parte, la labor de consultoría y dirección de proyecto se valora en 50 euros por hora, correspondiendo a un perfil Senior.

El tiempo estimado total para la implementación y documentación es de 300 horas para el desarrollador, desglosado en función de las agrupaciones lógicas de los Sprints planificados, a las que se suman 30 horas de revisión, resolución de dudas y dirección por parte del tutor.

Rol	Tarea / Fase (Sprints)	Horas	Coste / h	Total
Desarrollador	Investigación y Diseño (Sp 1-3)	50 h	20 euros	1.000,00 euros
Desarrollador	Core y Autenticación (Sp 4-7)	65 h	20 euros	1.300,00 euros
Desarrollador	Lógica y Tablero (Sp 8-12)	85 h	20 euros	1.700,00 euros
Desarrollador	Ajustes y Modos (Sp 13-15)	50 h	20 euros	1.000,00 euros
Desarrollador	Pruebas y Doc. (Sp 16-18)	50 h	20 euros	1.000,00 euros
Director Proy.	Consultoría y Revisión	30 h	50 euros	1.500,00 euros
Total Personal (330 h)				7.500,00 euros

Cuadro 3.2: Estimación desglosada de costes de personal.

3.3.2. Costes de Hardware

El desarrollo y las pruebas se realizarán con equipo propio. Para obtener su coste equivalente, se calculará la amortización del portátil y del móvil de pruebas durante los 9 meses del proyecto, asumiendo una vida útil de 4 años para cada dispositivo.

Equipo	Coste de compra	Meses de uso	Amortización
Ordenador portátil	1.200 euros	9	225,00 euros
Dispositivo móvil (Pruebas)	300 euros	9	56,25 euros
Total Hardware			281,25 euros

Cuadro 3.3: Estimación de costes de hardware (amortización).

3.3.3. Costes de Software

Gran parte de las herramientas elegidas para el proyecto tienen licencias gratuitas o de código abierto (Open Source), lo que reduce considerablemente esta partida. Adicionalmente, el videojuego resultante de la planificación y desarrollo de este proyecto será publicado y accesible de forma gratuita a través de su aplicación web oficial (<https://rentoy-online.web.app>).

Software / Servicio	Licencia / Plan	Coste Total
Flutter y Dart SDK	Open Source	0,00 euros
Visual Studio Code	Freeware	0,00 euros
Git y GitHub	Gratuito (Plan Básico)	0,00 euros
Firebase (Auth, RTDB, Firestore)	Plan Spark (Gratuito)	0,00 euros
Total Software		0,00 euros

Cuadro 3.4: Estimación de costes de software.

3.3.4. Coste Total del Proyecto

Al sumar todas las secciones anteriores obtenemos el presupuesto total estimado a priori para el desarrollo de *RentoyNow*.

Concepto	Total
Costes de Personal	7.500,00 euros
Costes de Hardware	281,25 euros
Costes de Software	0,00 euros
PRESUPUESTO TOTAL	7.781,25 euros

Cuadro 3.5: Presupuesto total estimado del proyecto.

3.4. Planteamiento del Problema

El principal desafío de este proyecto radica en desarrollar una aplicación móvil multijugador completa desde cero, abordando la digitalización de un juego de cartas tradicional. Esto incluye la implementación de un motor de reglas complejo, la gestión de múltiples usuarios, la sincronización de estados en tiempo real y el diseño de interfaces adaptadas a dispositivos móviles. Además, requiere la integración profunda de un framework de desarrollo móvil en nuestro caso vamos a utilizar Flutter, con servicios de infraestructura

en la nube para gestionar el guardado de datos y la comunicación en tiempo real para lo cual vamos a utilizar Firebase.

A continuación se detallan los principales retos técnicos y de diseño a los que se enfrenta el desarrollo de *RentoyNow*.

3.5. Arquitectura Multijugador y Sincronización en Tiempo Real

El desarrollo de un juego multijugador en tiempo real supone uno de los desafíos más importantes del proyecto. Al tratarse de un juego de cartas donde la información debe reflejarse instantáneamente en los dispositivos de 2, 4 o 6 jugadores, cualquier latencia o desincronización afectaría drásticamente a la jugabilidad.

Para resolver esto, se debe plantear una arquitectura basada en la nube que permita la comunicación bidireccional. El reto consiste en estructurar correctamente la base de datos para manejar la creación de salas (mediante un código PIN), la gestión de la sala de espera y la transmisión de eventos de la partida (reparto de cartas, jugar una carta, actualización de puntos). Se utilizarán tecnologías como *Firebase Realtime Database* para la comunicación de baja latencia durante la partida y *Cloud Firestore* para el almacenamiento persistente de las sesiones y puntuaciones globales.

3.6. Lógica del Juego y Motor de Reglas

El Rentoy es un juego tradicional que cuenta con un conjunto de reglas no triviales: jerarquía de cartas, el concepto de la "Muestra", la obligación de asistir al palo de salida (arrastre), y un sistema de cánticos.^o envites que interrumpe el flujo normal de las bazas.

El problema técnico radica en traducir estas reglas físicas en un modelo de estados controlable mediante código. El sistema debe ser capaz de:

- Generar y barajar un mazo de cartas de la baraja española (40 cartas).
- Repartir las cartas y extraer la muestra de forma aleatoria y sincronizada.

- Validar si una jugada es legal (por ejemplo, impidiendo jugar una carta aleatoria si el jugador está obligado a seguir el palo de arrastre).
- Gestionar el sistema de envites de forma asíncrona (cuando un equipo canta, el flujo se detiene a la espera de la respuesta del equipo contrario).
- Determinar qué carta gana cada baza evaluando su jerarquía y si coincide con la muestra.

3.7. Interfaces y Experiencia de Usuario

Más allá de la lógica interna, la calidad de la experiencia del usuario depende en gran medida de las interfaces visuales. Adaptar un juego de mesa en el que el jugador tiene un amplio campo visual a las reducidas dimensiones de un dispositivo móvil representa un reto de diseño y composición.

El proyecto requiere diseñar varias pantallas específicas:

- **Autenticación y Menú Principal:** Interfaces intuitivas para el acceso a la aplicación.
- **Sala de Espera (Lobby):** Un espacio visual donde los jugadores se mantienen a la espera de que se unan los demás jugadores y puedan escoger su equipo antes de iniciar la partida.
- **Tablero de Juego (HUD):** La interfaz más crítica. Debe mostrar de forma clara la mano del jugador, las cartas jugadas en la mesa, la carta de muestra, los marcadores de puntuación, botones jugables y historial de cartas usadas, todo ello respetando las dimensiones de un dispositivo.

3.8. Gestión de Usuarios y Autenticación

Finalmente, el proyecto necesita un sistema seguro y robusto para gestionar la identidad de los jugadores. El problema consiste en implementar un flujo de autenticación seguro (mediante correo electrónico con contraseña o mediante el proveedor de Google), y asociar esa identidad a un perfil de jugador en la base de datos que almacene su nombre de usuario y el avatar seleccionado.

3.9. Especificación de Requisitos

Para garantizar que el proyecto cumpla con los objetivos propuestos se han definido una serie de requisitos funcionales y no funcionales que el sistema final debe satisfacer.

3.9.1. Requisitos Funcionales

Los requisitos funcionales (RF) describen los comportamientos, acciones y flujos específicos que el sistema debe ser capaz de realizar [26].

- **RF-01 Gestión de Usuarios:** El sistema debe permitir el registro y autenticación de usuarios mediante correo electrónico/contraseña y proveedores de terceros (Google).
- **RF-02 Gestión de Perfil:** Cada usuario autenticado debe poder configurar su perfil, pudiendo cambiar de nombre de usuario, avatar y contraseña.
- **RF-03 Creación de Partidas:** Un usuario debe poder crear una nueva sala de juego, configurando el número máximo de jugadores y generando un código PIN numérico.
- **RF-04 Unión a Partidas:** Los usuarios deben poder unirse a una sala de juego existente introduciendo el PIN correspondiente.
- **RF-05 Sala de Espera:** El sistema debe mostrar los jugadores conectados en la sala de espera, permitirles elegir su equipo y permitir iniciar la partida una vez la sala este completa.
- **RF-06 Lógica de juego:** El sistema debe permitir a los jugadores usar sus cartas por turnos, validando que la jugada sea legal según las reglas del Rentoy y determinando el ganador de la ronda.
- **RF-07 Sistema de Envites:** Durante la partida, el sistema debe permitir a los equipos realizar `.envites`(cantar), notificar en pantalla al equipo contrario y procesar la aceptación o rechazo para ajustar la puntuación de la ronda.
- **RF-08 Puntuación y Rondas:** El sistema debe sumar automáticamente los puntos al final de cada ronda, vaciar el tablero, volver a repartir y determinar el equipo ganador cuando se alcance la puntuación límite de la partida.

- **RF-09 Ocultación de Cartas:** El sistema debe permitir a los jugadores ocultar su mano de cartas poniéndolas boca abajo en pantalla para mantener la privacidad frente a observadores.
- **RF-10 Historial de Bazas:** El sistema debe registrar las cartas que ya se han jugado en la ronda actual y ofrecer una interfaz para que el usuario pueda consultarlo.
- **RF-11 Consulta de Reglas:** La aplicación debe proporcionar una sección accesible donde se expliquen las normas oficiales, la jerarquía de las cartas y el funcionamiento de los cánticos.
- **RF-12 Modo Entrenamiento:** El sistema debe incluir un modo de práctica offline que permita al jugador enfrentarse a un oponente controlado por el sistema (bot) para familiarizarse con la interfaz y poder aprender las reglas del juego.

3.9.2. Requisitos No Funcionales

Los requisitos no funcionales (RNF) definen las restricciones arquitectónicas y los atributos de calidad del sistema [26].

- **RNF-01 Rendimiento y Latencia:** La propagación de eventos durante la partida (como lanzar una carta o hacer un envite) debe reflejarse en los dispositivos de los demás jugadores con la mínima latencia posible para no frenar el ritmo del juego.
- **RNF-02 Multiplataforma:** La aplicación debe ser compatible con los sistemas operativos móviles más extendidos (Android e iOS) desde una única base de código, utilizando el framework *Flutter*.
- **RNF-03 Adaptabilidad Visual:** La interfaz gráfica de usuario debe ser completamente responsiva, adaptándose correctamente a diferentes resoluciones, tamaños y orientaciones de pantallas de dispositivos móviles.
- **RNF-04 Seguridad de Datos:** El sistema en la nube debe implementar reglas de seguridad estructurales para impedir que usuarios no autorizados o malintencionados puedan modificar el estado de partidas ajenas o alterar los perfiles de otros jugadores.
- **RNF-05 Escalabilidad:** El backend del sistema (Firebase) debe ser capaz de soportar múltiples partidas simultáneas sin que el rendimiento general de la aplicación o la latencia se degraden.

- **RNF-06 Mantenibilidad:** El código fuente de la aplicación debe seguir una arquitectura modular clara que facilite la detección de errores y permita la actualización del sistema en el futuro.

4. DISEÑO DEL SISTEMA

4.1. Herramientas utilizadas

Para llevar a cabo el desarrollo de este proyecto se han empleado diversas herramientas tecnológicas, tanto para la escritura de código como para la gestión de base de datos y control de versiones. A continuación, se detallan las principales herramientas elegidas.

4.1.1. Flutter y Dart

Para el desarrollo de la aplicación cliente multiplataforma se ha utilizado Flutter, un *framework* de código abierto desarrollado por Google y lanzado en 2017. Flutter permite compilar de forma nativa para plataformas móviles, web y de escritorio desde una única base de código [27].

El lenguaje de programación base que emplea este *framework* es Dart, también creado por Google. Dart es un lenguaje optimizado para la creación de interfaces de usuario rápidas en cualquier plataforma, ofreciendo características como compilación *Just-In-Time* (JIT) para un ciclo de desarrollo ágil (gracias a la recarga en caliente o *Hot Reload*) y compilación *Ahead-Of-Time* (AOT) para un rendimiento óptimo en producción [28].

Se ha optado por esta tecnología en el proyecto RentoyNow por su enorme versatilidad a la hora de desplegar la aplicación simultáneamente en dispositivos Android y navegadores web, así como por la riqueza de su sistema de componentes visuales (*widgets*), que facilita el diseño de una interfaz reactiva.

4.1.2. Firebase

Para cubrir las necesidades de *backend* e infraestructura en la nube, se ha implementado Firebase, una plataforma de desarrollo de aplicaciones adquirida por Google en 2014 [29].

Dentro del ecosistema de Firebase, el proyecto hace uso intensivo de tres

servicios principales:

- **Firebase Authentication:** Utilizado para la gestión segura del registro e inicio de sesión de los usuarios.
- **Realtime Database:** Una base de datos NoSQL alojada en la nube que permite almacenar y sincronizar datos entre los usuarios en tiempo real. Este servicio es el núcleo tecnológico que sostiene la lógica multi-jugador del juego, sincronizando las jugadas, los turnos y el estado de la mesa instantáneamente.
- **Cloud Firestore:** Otra base de datos NoSQL más escalable, empleada en el proyecto para el almacenamiento persistente a largo plazo, como la información de los perfiles de los jugadores y sus historiales de partida.

4.1.3. Git y GitHub

Para el control de versiones y el respaldo del código, se ha recurrido a Git, un software de control de versiones distribuido diseñado por Linus Torvalds en 2005. Git permite llevar un registro exhaustivo de todos los cambios realizados en los archivos del proyecto, posibilitando la reversión de errores y facilitando la experimentación segura mediante el uso de ramas (*branches*) [30].

En conjunto con Git, se ha utilizado GitHub como plataforma de alojamiento remoto en la nube. GitHub ha funcionado como el repositorio principal y seguro para centralizar el código fuente y coordinar las distintas versiones de la aplicación a lo largo de los meses de desarrollo.

4.1.4. Jira Software

Jira es un software producido por la empresa Atlassian que permite la gestión de proyectos mediante el seguimiento de requerimientos, tareas, errores y metodologías ágiles [31].

Durante el desarrollo de este trabajo, se ha utilizado Jira como herramienta principal para gestionar el flujo de trabajo de Scrum. Se ha aprovechado especialmente para organizar las tareas del *Product Backlog*, gestionar los *Sprints* de forma estructurada.

4.1.5. Visual Studio Code

El entorno principal para la escritura y depuración del código ha sido Visual Studio Code (VS Code), un editor de código fuente desarrollado por Microsoft. Destaca por su alta capacidad de personalización mediante extensiones y su excelente integración nativa tanto con Flutter y Dart como con las herramientas de control de versiones [32].

4.1.6. LaTeX

Para la redacción, estructuración y maquetación final de esta memoria, se ha empleado LaTeX. Se trata de un sistema de composición de textos orientado especialmente a la creación de documentos académicos y científicos de alta calidad tipográfica [33]. Su uso ha garantizado un formato riguroso, automatizando la generación de índices, la gestión de la bibliografía cruzada y el formateo de las tablas del *Product Backlog* y del presupuesto.

4.2. Arquitectura de Software y Patrones de Diseño

Para abordar la complejidad técnica del proyecto y asegurar el cumplimiento del requisito de mantenibilidad (RNF-06), se ha prestado especial atención a la organización del código y a la aplicación de patrones de diseño de software reconocidos [34].

4.2.1. Estructura Modular del Proyecto

En el ecosistema de desarrollo móvil multiplataforma, la organización estructural del código es vital a medida que el proyecto escala. En este trabajo, se ha optado por una aproximación híbrida basada en la arquitectura orientada a características o *Feature-First* [35], combinada con una separación transversal por capas. El proyecto se divide en los siguientes directorios principales:

- **app:** Contiene la configuración inicial y el punto de entrada global de la aplicación.

- **core:** Agrupa todos los elementos transversales que son compartidos por múltiples características del juego. En su interior se definen las constantes globales, los *widgets* reutilizables (como botones genéricos y componentes de interfaz) y los **servicios** puros (como la conexión con Firebase).
- **features:** Es el núcleo de la arquitectura *Feature-First*. En lugar de organizar el código por tipo de archivo técnico (juntar todas las pantallas de la app en un lado y todos los controladores en otro), se organiza por áreas de funcionalidad (por ejemplo: *auth*, *sala_espera*, *partida*). Cada característica es completamente independiente y encapsula sus propias pantallas (*screens*), controladores y *widgets* específicos que solo le pertenecen a ella.
- **models:** Centraliza las clases de representación de datos abstractos (el modelo de dominio del negocio), tales como **Usuario**, **Partida**, **Carta** o las propias **Reglas**.

Esta estructura favorece la segregación de responsabilidades y permite escalar el proyecto sin que el código termine fuertemente acoplado.

4.2.2. Patrones de Diseño Aplicados

A nivel de ejecución, la aplicación de patrones de arquitectura y de diseño es fundamental para estructurar el manejo del estado, coordinar la interfaz gráfica y gestionar los datos en tiempo real de manera escalable [36]:

- **Model-View-ViewModel (MVVM) y Separación de Capas:** Siguiendo la guía oficial de arquitectura de Flutter, el proyecto aplica el principio de separación de responsabilidades dividiendo el sistema en capas. La **Capa de UI** (*screens* y *widgets*) se encarga estrictamente de la presentación, mientras que la **Capa de Datos** gestiona la lógica. Las clases de **controllers** actúan como *ViewModels*, transformando los datos asíncronos y actualizando el estado de forma unidireccional hacia las vistas [37].
- **Patrón Singleton para Servicios:** En el directorio **core/servicios**, clases críticas como **AuthService** o el servicio de base de datos están diseñadas bajo el patrón Singleton. Se inician con un constructor privado y exponen una única instancia estática. Esto garantiza que la aplicación mantenga una única conexión a Firebase y comparta la misma sesión durante todo el ciclo de vida, evitando colisiones.

- **Patrón Factory Method (Fábrica):** Aplicado extensamente en el directorio `models`. Constructores como `factory Carta.fromMap` actúan como fábricas capaces de construir y devolver instancias complejas de objetos a partir de los datos crudos (diccionarios o JSON) que llegan desde la base de datos, encapsulando la lógica de validación de variables.
- **Patrón Service Layer / Enrutador de datos:** La carpeta de servicios funciona como una puerta de enlace exclusiva hacia la nube. Los controladores de las pantallas nunca realizan llamadas directas a la base de datos; delegan esta responsabilidad a la capa de servicios. Esta abstracción hace que los cambios en el origen de los datos no afecten a la interfaz.

4.3. Modelo de Datos y Backend

Dado que el juego requiere tanto el almacenamiento de perfiles de usuario como la sincronización constante de la mesa de juego, se ha optado por dividir la responsabilidad del *backend* en dos bases de datos del ecosistema de Firebase, aplicando diferentes niveles de seguridad y estructuras.

Para visualizar y comprender mejor esta distribución, se presenta a continuación un diagrama estructural de ambas bases de datos:

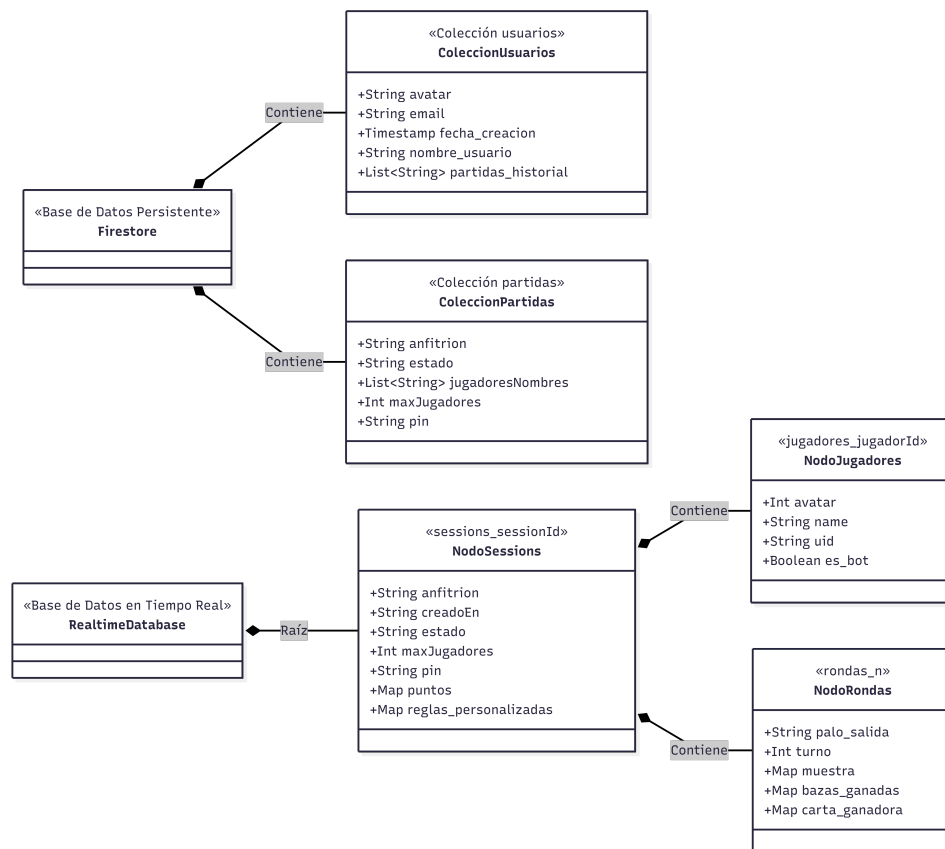


Figura 4.1: Diagrama de la Base de Datos (Estructura de Firestore y Realtime Database).

4.3.1. Almacenamiento Persistente (Cloud Firestore)

Cloud Firestore es una base de datos NoSQL orientada a documentos que se ha utilizado para guardar la información que debe perdurar en el tiempo. El modelo de datos se divide en dos colecciones principales:

- **Colección usuarios:** Almacena el perfil de cada jugador registrado. Cada documento tiene como identificador el UID generado por Firebase Authentication. Contiene campos como el **email**, el **nombre_usuario**, el **avatar** seleccionado, la **fecha_creacion** y un arreglo (*array*) denominado **partidas_historial** que guarda las referencias a las sesiones en las que el usuario ha participado. Para garantizar la privacidad y seguridad, las reglas de Firestore establecen que los perfiles pueden ser

leídos por cualquier persona (necesario para ver el nombre y avatar de los rivales), pero la escritura está restringida exclusivamente al propio dueño del documento (`request.auth.uid == userId`).

- **Colección partidas:** Almacena un registro de las partidas creadas. Contiene el `pin` de la sala, el identificador del `anfitrión`, el número máximo de jugadores y una lista con los nombres de los participantes (`jugadoresNombres`). Las reglas de seguridad exigen que un usuario deba estar autenticado para poder leer o escribir en esta colección. Esto permite que un anfitrión pueda crear o cancelar la sala, y que el resto de jugadores puedan unirse y actualizar los estados al finalizar, evitando accesos anónimos.

4.3.2. Motor Multijugador (Realtime Database)

Mientras que Firestore es ideal para datos persistentes, Firebase Realtime Database está optimizada para lecturas y escrituras de muy baja latencia. Por ello, se ha utilizado exclusivamente como el “motor” del juego en vivo.

La estructura se centraliza en el nodo `sessions`, donde cada identificador único aloja el estado completo de una partida activa en formato JSON (para consultar ejemplos visuales y completos de esta estructura y la de Firestore, véase el Apéndice B). Este árbol de datos contiene toda la información necesaria para reconstruir el tablero y sincronizar a los clientes, estructurada de la siguiente manera:

- **Metadatos de la sala:** Campos como el `pin` de acceso, el `anfitrión`, el `estado` (que transiciona de ‘esperando’ a ‘jugando’) y las `reglas personalizadas` de las cartas especiales.
- **Gestión de Jugadores y Marcador:** Un nodo `jugadores` que identifica a los participantes (nombre, UID y avatar), soportando además la integración de oponentes controlados por la máquina mediante el indicador `es_bot`. A la par, el nodo `puntos` lleva el recuento en tiempo real de ambos equipos y el `objetivo` final (ej. 21 tantos).
- **Motor de Rondas (rondas):** Es el núcleo lógico del juego. Se divide por cada iteración del reparto (1, 2, etc.) y mantiene un registro del `turno` actual. Dentro de cada ronda se sincronizan en vivo las cartas de cada jugador (marcando con el flag `usada` si ya se han tirado a la mesa), la carta que rige el triunfo (`muestra`), el `palo_salida` de la baza en curso, y un historial de las `bazas_ganadas` junto a la `carta_ganadora` de cada lance.

Para que el sistema de *matchmaking* (búsqueda de partidas) funcione correctamente y de forma segura, se han configurado reglas específicas. Aunque se exige autenticación para acceder a cualquier sesión, se ha habilitado un permiso especial de lectura indexada (`.indexOn: ["pin", "creadoEn"]`). Gracias a esto, un usuario puede consultar la base de datos buscando una sala que coincida exactamente con el PIN que ha introducido, sin que Firebase tenga que descargar el resto de partidas activas del servidor, optimizando así el rendimiento.

4.3.3. Entidades del Dominio (Modelos en Dart)

Para interactuar de forma segura con la base de datos y manejar la lógica del juego, la aplicación define una serie de entidades de dominio (Modelos) mediante clases en Dart. Estas clases utilizan constructores de tipo *Factory* para serializar y deserializar la información:

- **UsuarioModel y PartidaModel:** Mapean la información persistente de Firestore. El modelo de usuario incluye métodos auxiliares como `esSuTurno()`, facilitando la comprobación de turnos.
- **SesionModel:** Es la representación en objetos Dart del extenso árbol JSON que compone el motor multijugador en Realtime Database.
- **Carta y Baraja:** Representan los elementos físicos del juego. El modelo **Carta** es capaz de autocalcular la ruta de su recurso gráfico (*asset* PNG) deduciéndolo a partir de su número y su palo. El modelo **Baraja** inicializa la colección de 40 cartas, retirando los ochos y nueves según la normativa de la baraja española.

4.4. Diseño de Flujos y Secuencias

A nivel funcional, el sistema coordina a múltiples usuarios mediante flujos asíncronos. A continuación se diseñan y detallan mediante diagramas de secuencia los procesos más críticos del juego.

4.4.1. Flujo de Autenticación

El siguiente diagrama ilustra el flujo de inicio de sesión a través de Google OAuth. El cliente solicita las credenciales al proveedor, Firebase Authentication valida el token y, si el usuario es nuevo, el sistema crea automáticamente su documento en la colección `/usuarios` de Cloud Firestore con los valores por defecto (avatar 1 y marca de tiempo del servidor).

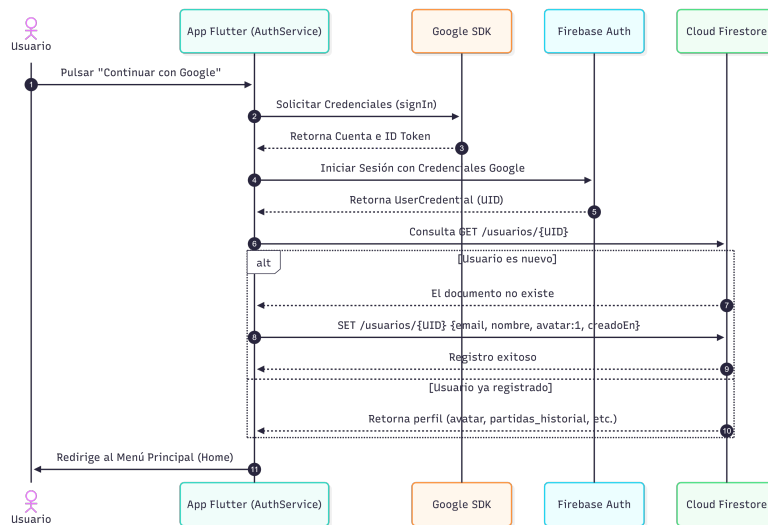


Figura 4.2: Diagrama de Secuencia del flujo de autenticación con Google y sincronización en Cloud Firestore.

4.4.2. Flujo de Matchmaking

El diagrama siguiente muestra cómo el anfitrión crea una sala y cómo un invitado se une a ella mediante PIN. Destacan la validación de unicidad del código, la transacción atómica para asignar el hueco al jugador entrante y la sincronización en tiempo real de la Sala de Espera hasta que el anfitrión arranca la partida.

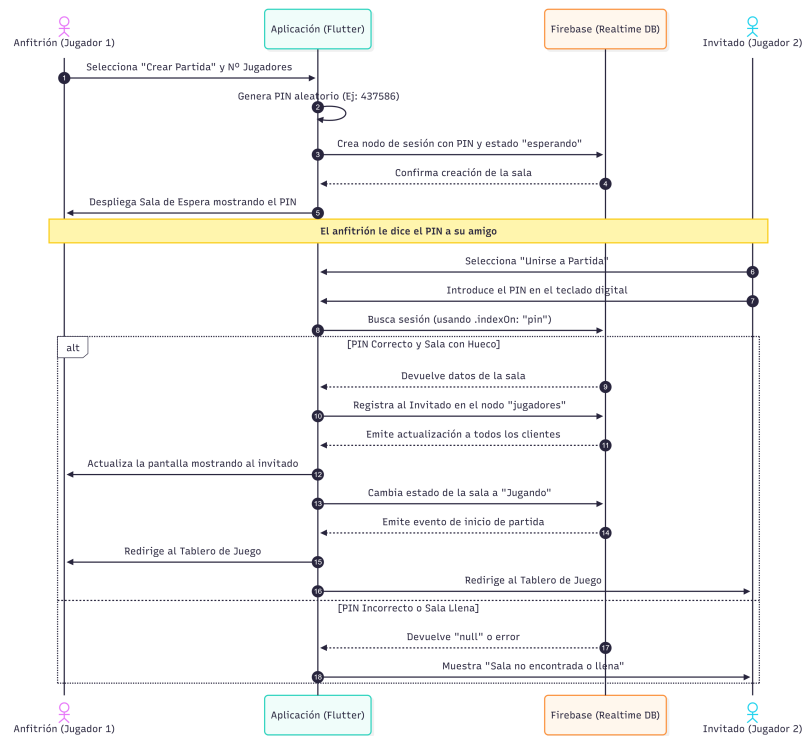


Figura 4.3: Diagrama de Secuencia del flujo de Matchmaking (Creación y entrada por PIN).

4.4.3. Diseño del Motor de Rondas y Bazas

El diseño del ciclo de vida de una partida, desde el reparto inicial de cartas hasta la resolución de cada ronda y la asignación del turno, se ha modelado como un proceso iterativo altamente acoplado a la base de datos para asegurar el sincronismo.

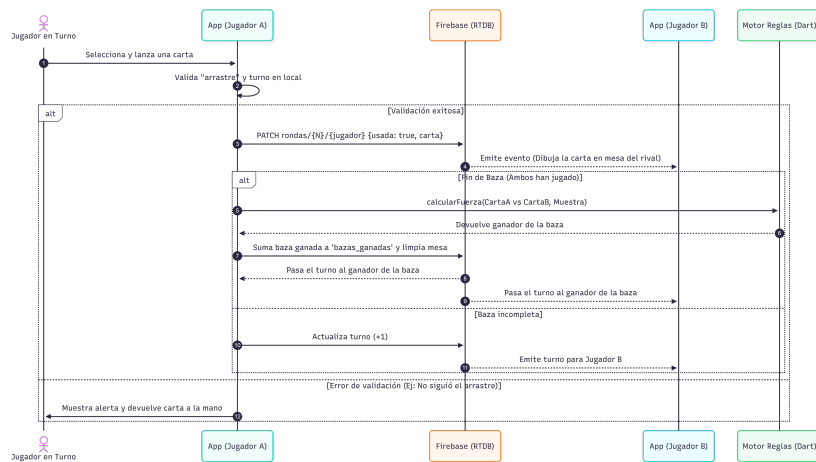


Figura 4.4: Diagrama de Secuencia del lanzamiento de cartas y cálculo asíncrono de baza.

4.4.4. Diseño del Sistema de Envites (Apuestas)

Las mecánicas comunicativas del Rentoy (cánticos y apuestas) se han diseñado como interrupciones asíncronas en el flujo de juego normal. Cuando un jugador canta, el flujo se detiene para el resto de clientes hasta que se emite una respuesta validada por el servidor.

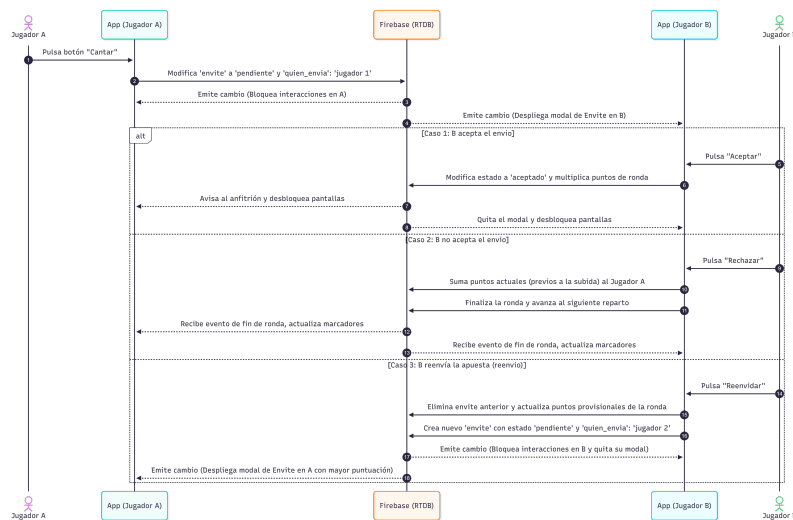


Figura 4.5: Diagrama de Secuencia del Sistema de Envites (Proceso de apuestas dinámicas).

5. RESOLUCIÓN E IMPLEMENTACIÓN

5.1. Lógica Interna y Mecánicas del Juego

Más allá del diseño visual y arquitectónico expuesto en el capítulo anterior, el correcto funcionamiento de *RentoyNow* recae sobre un robusto motor lógico programado en Dart. Este motor se encarga de traducir las reglas tradicionales del Rentoy a algoritmos evaluables de forma automática, gestionando el estado global de la partida en tiempo real.

5.1.1. Reparto y Gestión de Turnos

Al inicializar una partida, el `ControladorPartida` instancia el modelo `Baraja`, que genera las 40 cartas de la baraja española (sin ochos ni nueves) y las mezcla de forma aleatoria. Se reparten tres cartas a cada jugador y se extrae una carta adicional del mazo que actúa como **Muestra**, determinando el palo de triunfo para esa ronda. Toda esta información se serializa y se escribe en el nodo `rondas/{N}` de Realtime Database de forma atómica, garantizando que todos los clientes arranquen exactamente desde el mismo estado.

El turno de apertura de cada ronda se calcula de forma rotatoria: para la ronda N , el jugador inicial es $(N - 1)$ mód $maxJugadores + 1$, distribuyendo así el derecho de salida de forma equitativa a lo largo de la partida. Cuando un jugador gana una baza, la lógica le otorga el turno de apertura de la siguiente baza directamente, tal y como dictan las reglas del Rentoy.

5.1.2. Algoritmo de Evaluación de Bazas

El corazón lógico del juego reside en la jerarquía de clases abstractas `ReglasJuego` (`reglas_juego_model.dart`). Cuando un jugador juega su carta, el controlador ejecuta en local el método `calcularFuerza()` para determinar si la nueva carta supera a la actualmente ganadora. El algoritmo opera en tres niveles de prioridad:

1. **Cartas Especiales:** El método `getFuerzaCartaEspecial()` se evalúa primero. En la modalidad de 4 jugadores, el Tuerto (11 de Oros) obtiene una fuerza de 400 puntos. En 6 jugadores, la Perica (10 de Oros) es la más fuerte (400 pts), seguida del Pablo (5 de Oros, 390 pts) y el Tuerto (380 pts), formando una jerarquía exclusiva para partidas grandes.
2. **Bonificación de Categoría:** Si la carta no es especial, recibe una puntuación base de 200 si pertenece al palo de la Muestra (triumfo), de 100 si coincide con el `palo_salida` (la primera carta lanzada en la baza), o de 0 en cualquier otro caso.
3. **Jerarquía Interna:** Dentro de cada categoría, se aplica la posición de la carta en la lista de jerarquía. En partidas de 2 jugadores, para el palo de triunfo el orden es: 2, 12 (Rey), 11 (Caballo)... En partidas de 4 o más, el 3 desplaza al 2 como carta más fuerte del triunfo.

Gracias al patrón *Factory* implementado en `ReglasFactory`, el sistema selecciona automáticamente la clase de reglas correcta (`ReglasDosJugadores`, `ReglasCuatroJugadores`, `ReglasSeisJugadores` o `ReglasPersonalizadas`) en función del tamaño de la partida y de si el anfitrión configuró reglas personalizadas.

5.1.3. Implementación del Sistema de Envites (Apuestas)

Una de las mecánicas más complejas y características del Rentoy es la capacidad de realizar apuestas durante el transcurso de una ronda, conocido tradicionalmente como “cantar”. El `ControladorPartida` implementa este sistema de forma asíncrona mediante los métodos `cantar()`, `responderCantar()` y `rechazarCantar()`:

- **Lanzar el envite:** Un jugador puede proponer subir el valor de la ronda (por ejemplo, de 1 punto a 3). Esto actualiza el estado de la base de datos marcando el envite como **pendiente** y bloquea las jugadas regulares hasta que se resuelva.
- **Aceptar o Subir:** El rival puede aceptar la subida (la ronda continúa y el ganador se llevará la nueva cantidad) o rechazarla enviando un contra-envite (se vuelve a pasar el turno de respuesta al jugador original, subiendo aún más el valor).
- **Rechazar (Retirarse):** Si un equipo rechaza el envite, el motor finaliza automáticamente la ronda de forma prematura. El equipo que

lanzó la apuesta se anota los puntos que valía la ronda **antes** de la subida y se procede al siguiente reparto de cartas, validando de paso si este incremento de puntos supera el objetivo final de 21 tantos.

5.1.4. Inteligencia Artificial (Modo Bot)

Para ofrecer una experiencia de juego ininterrumpida sin necesidad de rivales humanos, se ha implementado un oponente artificial en el archivo `controlador_bot.dart`. El bot opera como `jugador 2` y se activa en el modo “Práctica” del menú principal.

El `ControladorBot` abre una escucha reactiva (*stream*) sobre el nodo de la sesión en Firebase. En cada cambio de estado, el método `_evaluarEstado()` decide la acción a realizar:

- **Gestión de Envites:** Si detecta un envite pendiente dirigido a él (`quien_responde == 'jugador 2'`), evalúa su mano. Si no tiene ninguna carta buena (triunfos o cartas especiales de Oros), rechaza la apuesta; en caso contrario, la acepta, replicando un comportamiento de riesgo básico.
- **Selección de Carta:** Si es su turno, selecciona la carta a jugar aplicando la misma lógica de evaluación de fuerza que un jugador humano: respeta la obligación del arrastre, intenta superar la carta ganadora actual si responde en la baza, o elige la carta más débil si abre baza sin estrategia clara. Una vez decidida, introduce un retardo artificial de 2 segundos para simular “pensamiento” y realiza exactamente la misma llamada a `_servicio.jugarCarta()` que realizaría un jugador humano.

Esta arquitectura garantiza que el bot no necesita ninguna ruta especial en el *backend*: reutiliza íntegramente la capa de servicios y las reglas del juego, por lo que hereda de forma transparente cualquier mejora futura en la lógica de validación.

6. RESULTADOS Y PRUEBAS

6.1. Resultados de la Interfaz de Usuario (UI)

El resultado visual final de *RentoyNow* demuestra el cumplimiento de los requisitos de usabilidad, ofreciendo una experiencia de usuario (UX) fluida tanto en dispositivos móviles como en navegadores web. El sistema se divide en distintas pantallas modulares que guían al usuario desde que abre la aplicación hasta que termina de jugar. A continuación, se detallan los resultados de estas interfaces.

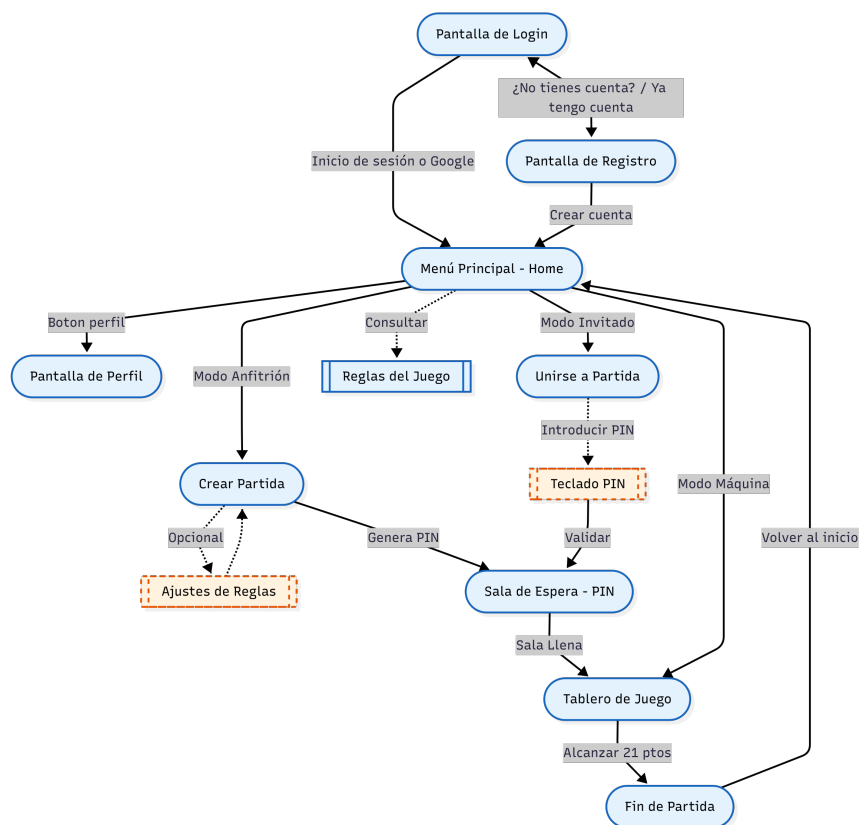


Figura 6.1: Diagrama de Flujo de Navegación e interfaces del sistema (UX Flow).

6.1.1. Autenticación, Menú Principal y Perfil

La puerta de entrada a la aplicación son las pantallas de **Inicio de Sesión** (*pantalla_login.dart*) y de **Registro** (*pantalla_registro.dart*). El sistema ofrece una doble vía de acceso: un formulario clásico de correo electrónico y contraseña, y un botón unificado de “Continuar con Google” que invoca el flujo nativo de autenticación OAuth.

Una vez autenticado, el usuario accede al **Menú Principal** (*pantalla_home.dart*), que actúa como panel de control central. El widget `MenuPrincipal` renderiza cuatro botones de acción: **Crear Partida**, **Unirse a Partida**, **Reglas del Juego** y **Práctica contra Bot**. Cada uno de ellos despliega un diálogo modal (*Dialog*) sin necesidad de navegar a una pantalla nueva, manteniendo el contexto del menú en todo momento.

Desde la barra de navegación superior, el usuario puede acceder a su **Pantalla de Perfil**, donde se muestran sus datos personales cargados desde Firestore.

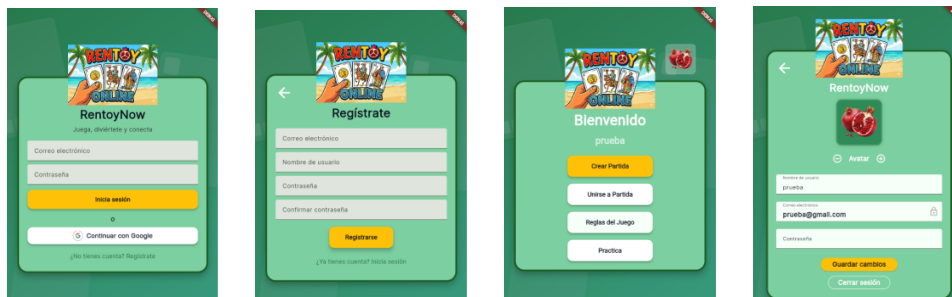


Figura 6.2: De izquierda a derecha: Inicio de Sesión, Registro, Menú Principal y Perfil del Jugador.

6.1.2. Matchmaking: Creación, Reglas y Búsqueda de Salas

Para iniciar una partida multijugador, la aplicación implementa un sistema de *matchmaking* basado en salas privadas identificadas por un PIN de seis dígitos.

Cuando el usuario pulsa **Crear Partida**, se despliega un diálogo que permite seleccionar el número de jugadores (2, 4 o 6). Este mismo modal incorpora una sección expandible de **Reglas Personalizadas**, mediante la cual el anfitrión puede reasignar el rango de prioridad de las cartas especiales.

Por su parte, el usuario que pulsa **Unirse a Partida** ve un teclado numérico digital para introducir el PIN del anfitrión.

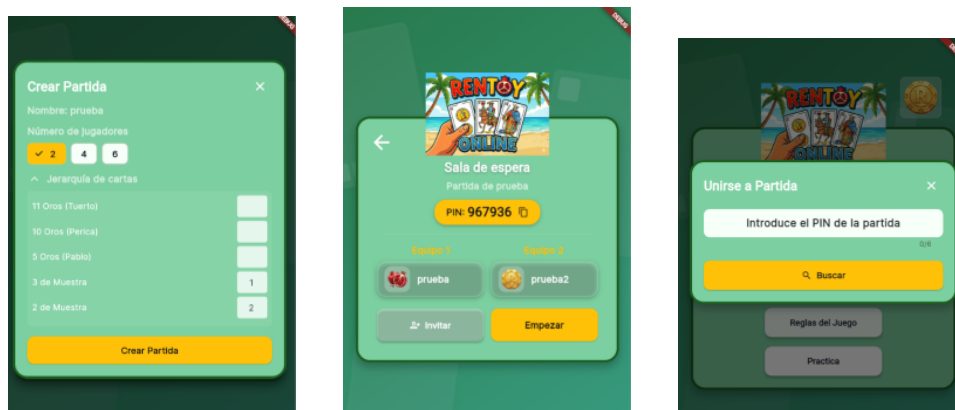


Figura 6.3: Modal de creación de partida con reglas personalizadas, Sala de Espera y teclado para unirse por PIN.

6.1.3. El Tablero de Juego y Elementos de Apoyo

La interfaz del **Tablero** (*pantalla_partida.dart*) es el núcleo visual y funcional de la aplicación. En la zona inferior se despliegan las cartas del jugador (Mano Interactiva). En la zona central se muestra la carta ganadora de la baza en curso junto a la Muestra (Mesa de Juego). En la zona superior se muestran los avatares de los rivales y el marcador de puntos por equipos.

El modal de **Reglas del Juego** es accesible desde el Menú Principal. Detalla las normas básicas del Rentoy. Finalmente, la **Pantalla de Fin de Partida** declara al equipo ganador y ofrece un botón para volver al Menú Principal.

Además, el tablero incorpora el **Modal de Envite** (con opciones de Querer, No Querer o Reenvitar) y el **Historial de Cartas**, que permite visualizar las cartas ya jugadas en rondas complejas.

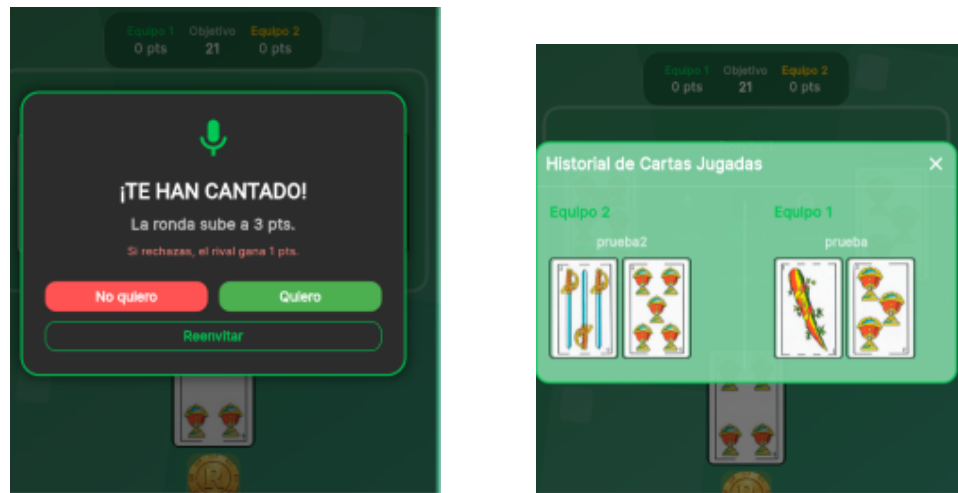


Figura 6.4: Modal de Envite con opciones de respuesta (izq.) y Popup de Historial de cartas jugadas (der.).



Figura 6.5: Tapete interactivo de juego, modal de Reglas del Juego y pantalla de Fin de Partida.

6.2. Pruebas y Validación

Para garantizar la estabilidad y jugabilidad del sistema multijugador, se han llevado a cabo diversas pruebas operativas en entornos controlados y reales.

6.2.1. Pruebas de Sincronización (Multijugador Real)

Se realizaron pruebas de estrés conectando múltiples dispositivos simultáneamente a una misma sala (simulando una partida completa al máximo de jugadores). Las lecturas y escrituras en Firebase Realtime Database mantuvieron una latencia mínima, ofreciendo una experiencia en tiempo real sin desincronizaciones ni bloqueos de turnos incluso en conexiones de red 4G estándar.

6.2.2. Pruebas de Lógica (Testing Interno)

El motor de validación de reglas (`ReglasFactory` y algoritmos de evaluación de bazas) fue probado frente a los casos límite más complejos del Rentoy, como el arrastre obligado y el cálculo de la carta ganadora bajo triunfos excepcionales (la Perica, el Tuerto). El comportamiento automatizado ha demostrado una alta precisión en la asignación de puntos, emulando exitosamente el criterio de un árbitro humano experto.

6.2.3. Pruebas de Interfaz (UI/UX)

Se ha validado la responsividad del tablero de juego interactivo tanto en pantallas móviles de distintos tamaños (desde 5.5 pulgadas hasta 6.7 pulgadas) como en navegadores web en dispositivos de escritorio. Esto asegura que el tamaño de las cartas y la distribución de los avatares sigan siendo jugables y proporcionales en todos los formatos sin superponer elementos cruciales.

7. CONCLUSIONES

El principal objetivo de este proyecto era desarrollar un videojuego multi-jugador completo desde cero, capaz de digitalizar la experiencia del tradicional juego de cartas Rentoy. Como objetivo secundario, se planteó mejorar los conocimientos de desarrollo en Flutter y los servicios en la nube de Firebase, aplicando además los conocimientos de ingeniería del software adquiridos durante el grado. Como se ha evidenciado a lo largo de los capítulos, ambos objetivos se han cumplido con éxito. Se ha creado un videojuego multiplataforma plenamente funcional siguiendo una metodología ágil (Scrum), y se ha aprendido a manejar las herramientas necesarias para su sincronización en tiempo real.

En cuanto a las fortalezas del proyecto, cabe destacar la robustez de la arquitectura en la nube. La división del backend entre Firestore (para persistencia a largo plazo) y Realtime Database (para el estado volátil del tablero) ha permitido una latencia muy baja, algo crucial en los juegos de cartas. Además, la modularidad del código fuente ha facilitado separar la interfaz gráfica de la lógica de negocio, haciendo que el añadido de nuevas mecánicas fuera mucho más sencillo y que no comprometiera las funcionalidades ya desarrolladas. No obstante, también existen algunas limitaciones y dificultades. El desarrollo de un juego multijugador en red conlleva una complejidad considerable. La sincronización estricta de turnos, sumada a reglas tan específicas como el arrastre o los envites asíncronos, han supuesto un desafío técnico de alto nivel.

Como posibles ampliaciones futuras del proyecto, se pueden plantear varias mejoras enfocadas a expandir la jugabilidad y convertir la aplicación en un producto más competitivo y comercial. Una de las vías principales sería la introducción de un ecosistema de apuestas con premios reales, donde los jugadores pudieran aportar una cuota de entrada para competir por botes de dinero en metálico. A esto se le podría sumar la organización y participación en torneos estructurados oficiales. Además, la interfaz dinámica y modular de la aplicación facilita enormemente la incorporación de mayores opciones de personalización; se podría añadir una tienda virtual donde los usuarios pudieran desbloquear nuevos avatares, diseños exclusivos para los reversos de las cartas y tapetes de juego personalizados.

Aunque ya se poseía una base previa de Flutter adquirida durante el grado, el desarrollo de una arquitectura multijugador en tiempo real con Firebase suponía un terreno prácticamente inexplorado. Ha resultado muy

reconfortante y alentador ir dominando la integración de estas herramientas complejas a medida que la dificultad del proyecto lo requería. En una profesión como la ingeniería informática, en la que es fundamental mantenerse en aprendizaje continuo debido a la aparición constante de nuevos retos y tecnologías, la realización de este proyecto deja un buen sabor de boca y la confianza en que se han consolidado los conocimientos necesarios para continuar creciendo como profesional en el futuro.

Bibliografía

- [1] M. I. Villagra. Los naipes, un juego con mucha historia. La casa del Creador, 2024. Recuperado el 3 de noviembre de 2025, de <https://lacasadelrecreador.com/es/blog/169-los-naipes-un-juego-con-mucha-historia>.
- [2] M. I. Villagra Romero. Breve historia de los naipes. Historia y Arabismo, 2016. Recuperado el 3 de noviembre de 2025, de <https://historiayarabismo.wixsite.com/recreahistoria/single-post/2017/06/30/breve-historia-de-los-naipes>.
- [3] NH Fournier. El origen de los naipes de la baraja española, 2022. Recuperado el 4 de noviembre de 2025, de <https://www.nhfournier.es/blog/origen-baraja-espanola/>.
- [4] S. Rubio Alfonso. Historia de la baraja española; origen, simbología y evolución. Repositorio Institucional URJC, 2023. Recuperado el 4 de noviembre de 2025, de <https://hdl.handle.net/10115/26178>.
- [5] J. Torregrosa. La baraja española: historia, palos y juegos. 888casino, 2023. Recuperado el 5 de noviembre de 2025, de <https://www.888casino.es/blog/baraja-espanola>.
- [6] NH Fournier. Curiosidades de la baraja española, 2022. Recuperado el 5 de noviembre de 2025, de <https://www.nhfournier.es/blog/curiosidades-baraja-espanola/>.
- [7] Solitaire 365. Los 10 tipos de juegos de cartas, s.f. Recuperado el 6 de noviembre de 2025, de <https://www.solitaire365.com/es/estrategias/tipos-de-juegos-de-cartas>.
- [8] D. Winter. Magnavox odyssey: the first home video game console. Pong-Story, s.f. Recuperado el 7 de noviembre de 2025, de <https://www.pong-story.com/odyssey.htm>.
- [9] Digital Trends. Magnavox odyssey, la misteriosa primera consola del mundo, 2025. Recuperado el 7 de noviembre de 2025, de <https://es.digitaltrends.com/videojuego/magnavox-odyssey-primera-consola-videojuegos-historia-ralph-baer/>.
- [10] EBSCO Research Starters. History of video games, 2025. Recuperado el 6 de noviembre de 2025, de <https://www.ebsco.com/research-starters/history/history-video-games>.

-
- [11] ReparandoAndo. Game boy: la consola portátil que redefinió la industria, 2025. Recuperado el 9 de noviembre de 2025, de <https://reparandoando.com/blog/game-boy-historia.php>.
- [12] La Tercera. Game boy, la icónica consola portátil que revolucionó el mercado, 2025. Recuperado el 10 de noviembre de 2025, de <https://www.latercera.com/que-pasa/noticia/game-boy-consola-portatil-revoluciono-mercado/751815/>.
- [13] Gameopedia. The history, evolution, and future of mobile gaming, s.f. Recuperado el 10 de noviembre de 2025, de <https://gameopedia.com/blogs/the-history-evolution-and-future-of-mobile-gaming>.
- [14] J. Juul. *A casual revolution: Reinventing video games and their players*. MIT press, 2010. Recuperado el 14 de noviembre de 2025, de <https://www.jesperjuul.net/casualrevolution/>.
- [15] Udonis Blog. Mobile games: The complete history (1993–2025), s.f. Recuperado el 10 de noviembre de 2025, de <https://www.blog.udonis.co/mobile-marketing/mobile-games/mobile-games>.
- [16] C. Feijoo, J. L. Gómez-Barroso, J. M. Aguado, and S. Ramos. Mobile gaming: Industry challenges and policy implications. *Telecommunications Policy*, 36(3):212–221, 2012. Recuperado el 15 de noviembre de 2025, de <https://doi.org/10.1016/j.telpol.2011.12.004>.
- [17] AVPasión. Así ha sido la evolución de los juegos móviles, 2025. Recuperado el 10 de noviembre de 2025, de <https://www.avpasion.com/evolucion-juegos-moviles-comenzo-anos-60/>.
- [18] License Global. Mattel celebrates 50th anniversary of uno, 2021. Recuperado el 12 de noviembre de 2025, de <https://www.licenseglobal.com/toys-games/mattel-celebrates-50th-anniversary-uno>.
- [19] Vida Extra. Nació como hobby y su creador terminó ganando 1 000 000 dólares..., 2024. Recuperado el 12 de noviembre de 2025, de <https://www.vidaextra.com/estrategia/nacio-como-hobby-su-creador-termino-ganando-1-000-000-dolares-ocho-horas-as>.
- [20] Xataka. Balatro ha sido el último gran soplo de aire fresco en la industria..., 2024. Recuperado el 13 de noviembre de 2025, de <https://www.xataka.com/videojuegos/balatro-ha-sido-ultimo-gran-soplo-aire-fresco-industria-videojuegos-independ>.
- [21] Vandal. Balatro – análisis (pc), 2024. Recuperado el 13 de noviembre de 2025, de <https://vandal.elespanol.com/analisis/pc/balatro/141759#p-13>.

- [22] MobilityWare. Spades: Classic trick-taking card game. MobilityWare Official Website, s.f. Recuperado el 5 de mayo de 2026, de <https://www.mobilityware.com/games/spades/>.
- [23] EcuRed. La brisca: Historia y reglas. Enciclopedia Cubana, s.f. Recuperado el 5 de mayo de 2026, de <https://www.ecured.cu/Brisca>.
- [24] Quarzo Aplicaciones y Juegos. Juegos clásicos: La brisca. Quarzo Apps Official Website, s.f. Recuperado el 5 de mayo de 2026, de <https://quarzoapps.com/juegos/la-brisca/>.
- [25] Ken Schwaber and Jeff Sutherland. The scrum guide: La guía definitiva de scrum: Las reglas del juego, 2020. Recuperado el 11 de mayo de 2026, de <https://scrumguides.org/scrum-guide.html>.
- [26] Visure Solutions. Guía de requisitos funcionales frente a requisitos no funcionales, s.f. Recuperado el 7 de mayo de 2026, de <https://visuresolutions.com/es/alm-guide/functional-vs-non-functional-requirements/>.
- [27] Google. Flutter - build apps for any screen, 2026. Recuperado de <https://flutter.dev/>.
- [28] Google. Dart programming language, 2026. Recuperado de <https://dart.dev/>.
- [29] Google. Firebase, 2026. Recuperado de <https://firebase.google.com/>.
- [30] Software Freedom Conservancy. Git, 2026. Recuperado de <https://git-scm.com/>.
- [31] Atlassian. Jira software, 2026. Recuperado de <https://www.atlassian.com/software/jira>.
- [32] Microsoft. Visual studio code - code editing. redefined, 2026. Recuperado de <https://code.visualstudio.com/>.
- [33] The LaTeX Project. Latex – a document preparation system, 2026. Recuperado de <https://www.latex-project.org/>.
- [34] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. Información disponible en https://en.wikipedia.org/wiki/Design_Patterns.
- [35] A. Bizzotto. Flutter project structure: Feature-first or layer-first? Code with Andrea, 2021. Recuperado de <https://codewithandrea.com/articles/flutter-project-structure/>.

- [36] M. Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [37] Google - Flutter Team. Architecting flutter apps, 2024. Recuperado de <https://docs.flutter.dev/app-architecture>.

A. TARJETAS DE HISTORIAS DE USUARIO

En este apéndice se detallan exhaustivamente todas las historias de usuario que conforman el *Product Backlog* del proyecto RentoyNow. Para cada una de ellas se incluye su descripción funcional desde la perspectiva del usuario y sus respectivos criterios de aceptación, los cuales han guiado la validación técnica durante los *Sprints* de desarrollo documentados en la metodología.

A.1. Gestión de Usuarios

ID: HU-01	Nombre: Registro con email
Descripción: Como visitante, quiero registrarme con correo y contraseña para poder acceder al sistema.	
Criterios de Aceptación: ▪ El usuario puede introducir email y contraseña. ▪ El sistema valida formato del email. ▪ La contraseña debe cumplir requisitos mínimos. ▪ El sistema evita emails duplicados. ▪ Tras registrarse, el usuario queda autenticado.	

ID: HU-02	Nombre: Inicio de sesión
Descripción: Como usuario registrado, quiero iniciar sesión para acceder a mis partidas y perfil.	
Criterios de Aceptación: ▪ El usuario introduce email y contraseña. ▪ El sistema valida credenciales. ▪ Si son incorrectas, se muestra error. ▪ Si son correctas, accede al menú principal.	

ID: HU-03	Nombre: Inicio de sesión con Google
Descripción: Como usuario, quiero iniciar sesión con Google para acceder más rápido sin crear contraseña.	
Criterios de Aceptación: ▪ Existe botón “Continuar con Google”. ▪ El sistema solicita permisos de Google. ▪ El usuario accede correctamente tras autenticarse.	

ID: HU-04	Nombre: Recuperación de contraseña
Descripción: Como usuario, quiero recuperar mi contraseña para volver a acceder si la olvido.	
Criterios de Aceptación: ▪ El usuario puede solicitar recuperación. ▪ El sistema envía correo de restablecimiento. ▪ El usuario puede establecer nueva contraseña.	

A.2. Perfil

ID: HU-05	Nombre: Cambiar nombre de usuario
Descripción: Como usuario autenticado, quiero cambiar mi nombre para personalizar mi perfil.	
Criterios de Aceptación: ▪ El sistema permite editar el nombre. ▪ El nombre actualizado se refleja en partidas.	

ID: HU-06	Nombre: Cambiar avatar
Descripción: Como usuario, quiero cambiar mi avatar para identificarme visualmente.	
Criterios de Aceptación: ▪ El usuario puede subir imagen o elegir predeterminada. ▪ El sistema valida formato permitido. ▪ El avatar se actualiza correctamente.	

ID: HU-07	Nombre: Cambiar contraseña
Descripción: Como usuario autenticado, quiero cambiar mi contraseña para mejorar mi seguridad.	
Criterios de Aceptación: ▪ Debe introducir contraseña actual. ▪ La nueva contraseña se valida. ▪ El sistema confirma el cambio.	

A.3. Creación y Unión de Partidas

ID: HU-08	Nombre: Crear sala
Descripción: Como usuario, quiero crear una sala para jugar con otros jugadores.	
Criterios de Aceptación: ▪ El usuario puede crear sala. ▪ El sistema genera PIN único. ▪ La sala queda visible para jugadores invitados.	

ID: HU-09	Nombre: Configurar máximo de jugadores
Descripción: Como creador de sala, quiero definir el máximo de jugadores para controlar la partida.	
Criterios de Aceptación: ▪ El sistema permite elegir capacidad (ej. 2, 4 o 6 jugadores). ▪ No se permiten más jugadores de los establecidos.	

ID: HU-10	Nombre: Generar PIN
Descripción: Como sistema, quiero generar un PIN numérico único para identificar cada partida.	
Criterios de Aceptación: ▪ El PIN es numérico. ▪ No existen PIN repetidos activos.	

ID: HU-11	Nombre: Unirse mediante PIN
Descripción: Como usuario, quiero introducir un PIN para entrar en una partida existente.	
Criterios de Aceptación: ▪ El usuario puede escribir PIN. ▪ El sistema valida existencia de sala. ▪ Si el PIN es incorrecto, se muestra error.	

ID: HU-12	Nombre: Validar sala completa
Descripción: Como sistema, quiero impedir nuevas conexiones cuando la sala esté llena.	
Criterios de Aceptación: ▪ El sistema rechaza jugadores extra. ▪ Se muestra mensaje de sala completa.	

A.4. Sala de Espera

ID: HU-13	Nombre: Mostrar jugadores conectados
Descripción: Como jugador, quiero ver quién está conectado en la sala antes de comenzar.	
Criterios de Aceptación: ▪ La lista se actualiza en tiempo real. ▪ Se muestran nombres y equipos.	

ID: HU-14	Nombre: Elegir equipo
Descripción: Como jugador, quiero seleccionar mi equipo antes de iniciar la partida.	
Criterios de Aceptación: ▪ El jugador puede cambiar de equipo. ▪ El sistema evita desequilibrios inválidos.	

ID: HU-15	Nombre: Iniciar partida
Descripción: Como anfitrión, quiero iniciar la partida cuando todos estén preparados.	
Criterios de Aceptación: ▪ Solo el creador puede iniciar. ▪ La partida solo inicia con jugadores suficientes.	

ID: HU-16	Nombre: Mostrar estado de preparación
Descripción: Como jugador, quiero marcarme como listo para informar que puedo empezar.	
Criterios de Aceptación: ▪ Existe indicador “Listo”. ▪ Los demás jugadores ven el estado actualizado.	

A.5. Lógica de Juego

ID: HU-17	Nombre: Repartir cartas
Descripción: Como sistema, quiero repartir cartas automáticamente al iniciar cada ronda.	
Criterios de Aceptación: ▪ Todos reciben cartas válidas. ▪ Ninguna carta se repite en el tablero.	

ID: HU-18	Nombre: Jugar carta
Descripción: Como jugador, quiero jugar cartas durante mi turno para participar en la ronda.	
Criterios de Aceptación: ▪ Solo puede jugar el jugador activo. ▪ La carta aparece en la mesa.	

ID: HU-19	Nombre: Validar jugada legal
Descripción: Como sistema, quiero validar las jugadas para asegurar que cumplen las reglas de arrastre.	
Criterios de Aceptación: ▪ El sistema impide jugadas ilegales. ▪ Se informa del motivo del error.	

ID: HU-20	Nombre: Determinar ganador de baza
Descripción: Como sistema, quiero calcular automáticamente qué jugador gana la baza.	
Criterios de Aceptación: ▪ Se aplican reglas oficiales del Rentoy. ▪ El ganador obtiene el turno siguiente.	

ID: HU-21	Nombre: Controlar turnos
Descripción: Como sistema, quiero controlar el orden de turnos para mantener la partida organizada.	
Criterios de Aceptación: ▪ Solo un jugador tiene turno activo. ▪ El orden rota correctamente.	

A.6. Envites

ID: HU-22	Nombre: Realizar envite
Descripción: Como jugador, quiero cantar un envite para aumentar los puntos en juego.	
Criterios de Aceptación: ▪ Existe botón de envite. ▪ El sistema notifica al equipo rival.	

ID: HU-23	Nombre: Aceptar envite
Descripción: Como equipo rival, quiero aceptar un envite para disputar la apuesta.	
Criterios de Aceptación: ▪ El sistema registra aceptación. ▪ La puntuación en juego aumenta correctamente.	

ID: HU-24	Nombre: Rechazar envite
Descripción: Como equipo rival, quiero rechazar un envite para minimizar pérdidas.	
Criterios de Aceptación: ▪ El sistema asigna puntos correspondientes al equipo que canta. ▪ La partida continúa o se resuelve la baza.	

ID: HU-25	Nombre: Mostrar notificaciones de envite
Descripción: Como jugador, quiero ver claramente cuándo alguien canta para reaccionar rápidamente.	
Criterios de Aceptación: ▪ La notificación aparece en pantalla. ▪ Incluye botones para contestar.	

A.7. Puntuación y Rondas

ID: HU-26	Nombre: Calcular puntuación
Descripción: Como sistema, quiero sumar puntos automáticamente al finalizar la ronda.	
Criterios de Aceptación: ▪ La puntuación se actualiza correctamente. ▪ Los puntos son visibles para todos.	

ID: HU-27	Nombre: Reiniciar mesa
Descripción: Como sistema, quiero limpiar la mesa tras cada ronda para comenzar la siguiente.	
Criterios de Aceptación: ▪ Las cartas desaparecen de la mesa. ▪ Se inicia nuevo reparto.	

ID: HU-28	Nombre: Detectar ganador de partida
Descripción: Como sistema, quiero finalizar la partida cuando un equipo alcance la puntuación límite.	
Criterios de Aceptación: ▪ Se muestra equipo ganador. ▪ La partida queda cerrada.	

ID: HU-29	Nombre: Mostrar marcador
Descripción: Como jugador, quiero ver el marcador en todo momento para conocer el estado de la partida.	
Criterios de Aceptación: ▪ El marcador está visible permanentemente. ▪ Se actualiza en tiempo real.	

A.8. Privacidad de Cartas

ID: HU-30	Nombre: Ocultar mano
Descripción: Como jugador, quiero ocultar mis cartas para evitar que otros las vean en persona.	
Criterios de Aceptación: ▪ Existe opción “Ocultar cartas”. ▪ Las cartas se muestran boca abajo en el dispositivo.	

ID: HU-31	Nombre: Mostrar mano nuevamente
Descripción: Como jugador, quiero volver a mostrar mis cartas cuando lo necesite.	
Criterios de Aceptación: ▪ El jugador puede alternar ocultar/mostrar. ▪ La acción es instantánea.	

A.9. Historial de Bazas

ID: HU-32	Nombre: Consultar historial
Descripción: Como jugador, quiero consultar las cartas jugadas para recordar bazas anteriores.	
Criterios de Aceptación: ▪ Existe sección de historial en la UI. ▪ Se muestran cartas jugadas en orden.	

ID: HU-33	Nombre: Registrar cartas jugadas
Descripción: Como sistema, quiero almacenar automáticamente las cartas jugadas durante la ronda.	
Criterios de Aceptación: ▪ Todas las cartas quedan registradas. ▪ El historial es accesible durante la ronda.	

A.10. Consulta de Reglas

ID: HU-34	Nombre: Ver reglas oficiales
Descripción: Como usuario, quiero consultar las reglas para aprender a jugar correctamente.	
Criterios de Aceptación: ▪ Existe sección de reglas. ▪ Las reglas son accesibles desde menú.	

ID: HU-35	Nombre: Consultar jerarquía de cartas
Descripción: Como jugador, quiero ver el valor de las cartas para tomar mejores decisiones.	
Criterios de Aceptación: ▪ La jerarquía aparece claramente explicada. ▪ Incluye ejemplos visuales.	

ID: HU-36	Nombre: Consultar envites
Descripción: Como jugador, quiero entender cómo funcionan los cánticos para usarlos correctamente.	
Criterios de Aceptación: ▪ Se explican tipos de envites (Rentoy, Falta). ▪ Incluye puntuaciones asociadas.	

A.11. Modo Entrenamiento

ID: HU-37	Nombre: Iniciar modo entrenamiento
Descripción: Como jugador, quiero practicar contra un bot para aprender el juego.	
Criterios de Aceptación: ▪ Existe opción “Modo entrenamiento”. ▪ La partida funciona offline.	

ID: HU-38	Nombre: Jugar contra IA
Descripción: Como jugador, quiero enfrentarme a un bot para mejorar mis habilidades.	
Criterios de Aceptación: ■ El bot juega automáticamente. ■ Respetar reglas oficiales.	

ID: HU-39	Nombre: Aprender reglas durante práctica
Descripción: Como principiante, quiero recibir ayuda contextual mientras juego para aprender más rápido.	
Criterios de Aceptación: ■ El sistema muestra consejos. ■ Explica jugadas inválidas de manera clara.	

ID: HU-40	Nombre: Reiniciar entrenamiento
Descripción: Como jugador, quiero reiniciar una práctica para volver a intentarlo.	
Criterios de Aceptación: ■ Existe botón de reinicio. ■ Se reinicia estado completo de la partida.	

B. ESQUEMAS DE BASE DE DATOS

En este apéndice se detalla la estructura en formato JSON de las dos bases de datos utilizadas en el proyecto. Esta información complementa la explicación arquitectónica de los modelos de datos, permitiendo visualizar de forma clara cómo se almacena y organiza la información tanto efímera como persistente.

B.1. Realtime Database (Motor del Juego)

El siguiente bloque muestra la estructura completa de una partida en curso dentro del nodo **sessions**. Se puede apreciar cómo se organiza la información de dos jugadores reales, la puntuación de los equipos, y el registro detallado de las rondas y bazas que componen la lógica multijugador.

Listing B.1: Estructura JSON de una sesión en Realtime Database

```
1 {
2   "anfitrión": "prueba_user",
3   "creadoEn": "2026-05-14T14:40:04.031",
4   "estado": "jugando",
5   "maxJugadores": 2,
6   "pin": "437586",
7   "puntos": {
8     "equipo1": 1,
9     "equipo2": 0,
10    "objetivo": 21
11  },
12  "reglas_personalizadas": {
13    "2_muestra": 2,
14    "3_muestra": 1
15  },
16  "jugadores": {
17    "jugador 1": {
18      "avatar": 2,
19      "name": "prueba_user",
20      "uid": "QoNj8QaSwSTZXmK4JJyD9A0vTuz1"
21    },
22    "jugador 2": {
23      "avatar": 5,
24      "name": "rival_real",
25      "uid": "PqLz9WbTxRCYxLp3KKaF8B1wMpQ2"
26    }
27  },
28  "rondas": {
29    "actual": 2,
```

```
30  "1": {
31    "palo_salida": "Espadas",
32    "puntos": 1,
33    "turno": 1,
34    "muestra": {
35      "numero": 11,
36      "palo": "Espadas",
37      "usada": false
38    },
39    "bazar_ganadas": {
40      "baza1": "jugador 1",
41      "baza2": "jugador 1"
42    },
43    "carta_ganadora": {
44      "carta": { "numero": 10, "palo": "Espadas", "usada": false },
45      "equipo": 1,
46      "jugador": "jugador 1"
47    }
48  },
49  "2": {
50    "palo_salida": "Bastos",
51    "puntos": 1,
52    "turno": 1,
53    "muestra": {
54      "numero": 6,
55      "palo": "Espadas",
56      "usada": false
57    },
58    "jugador 1": [
59      { "numero": 10, "palo": "Bastos", "usada": false },
60      { "numero": 11, "palo": "Bastos", "usada": false },
61      { "numero": 3, "palo": "Bastos", "usada": true }
62    ],
63    "jugador 2": [
64      { "numero": 10, "palo": "Copas", "usada": false },
65      { "numero": 7, "palo": "Bastos", "usada": true },
66      { "numero": 2, "palo": "Oros", "usada": true }
67    ],
68    "bazar_ganadas": {
69      "baza1": "jugador 2"
70    },
71    "carta_ganadora": {
72      "carta": { "numero": 7, "palo": "Bastos", "usada": false },
73      "equipo": 2,
74      "jugador": "jugador 2"
75    }
76  }
77 }
78 }
```

B.2. Cloud Firestore (Almacenamiento Persistente)

Firestore se utiliza para persistir la información de los perfiles de usuario y el historial de las partidas creadas en el juego. A continuación se exponen

ejemplos simplificados de documentos pertenecientes a las dos colecciones principales.

B.2.1. Colección: usuarios

Listing B.2: Estructura JSON de un documento en la colección usuarios

```
1 {
2   "avatar": 2,
3   "email": "prueba@gmail.com",
4   "fecha_creacion": {
5     "_seconds": 1712595210,
6     "_nanoseconds": 0
7   },
8   "nombre_usuario": "prueba",
9   "partidas_historial": [
10    "-0saV42DLL07MliT3BFE",
11    "-0saXY_be_xLzyNne-cb"
12  ]
13 }
```

B.2.2. Colección: partidas

Listing B.3: Estructura JSON de un documento en la colección partidas

```
1 {
2   "anfitrión": "prueba_user",
3   "estado": "finalizada",
4   "jugadores": [
5     "prueba_user",
6     "rival_real"
7   ],
8   "maxJugadores": 2,
9   "pin": "437586"
10 }
```